action

The action namespace is produced by one regex pass over one combined text string: tweet body + OCR text + transcript text + media-description text, joined with separators. A tag is added when its regex appears anywhere in that combined string.

Tag	Count	Precise plain-English rule
action:detention	2,278 (22.8%)	Matches arrest/arrested/arresting, detain/detained/detaining, apprehend/apprehended/apprehending, in custody, or nab/nabbed.
action:deportation	1,304 (13.0%)	Matches deport/deported/deporting/deportation, remove/removed/removal, or repatriated/repatriation, but the deport match is guarded so self-deport does not count here.
action:self-deportation	191 (1.9%)	Matches self-deport/self deport variants, voluntary/voluntarily departure/departed/departing, leave voluntarily, report <your/their/his/her/an/the> departure, or take control of <your/their/his/her> departure.
action:report-immigrants	7 (0.1%)	Matches report/call/tip language only when ICE/DHS/contact information and immigrant/alien/migrant/immigration-violation subject language occur in the same bounded expression. The bounds in the regex are 80, 120, 180, or 240 characters depending on phrase order.

Code text

scripts/tag_lexical.py:515-559

```
515: PATTERN_FRAME_CRIMINAL = _compile(
516:     r"\b(criminal illegal aliens?|illegal aliens?|criminal aliens?|aggravated felons?|convicted (?:for|of))\b"
517: )
518: PATTERN_ACTION_DETENTION = _compile(
519:     r"\b(arrest(?:ed|ing)?|detain(?:ed|ing)?|apprehend(?:ed|ing)?|in custody|nab(?:bed)?)\b"
520: )
521: PATTERN_ACTION_SELF_DEPORTATION = _compile(
522:     r"\bself[- ]deport(?:ed|ing|ation)?\b"
523:     r"|\bvoluntar(?:y|ily)\s+depart(?:ure|ed|ing)?\b"
524:     r"|\bleave voluntarily\b"
525:     r"|\breport\s+(?:your|their|his|her|an|the)?\s*departure\b"
526:     r"|\btake\s+control\s+of\s+(?:your|their|his|her)\s+departure\b"
527: )
528: PATTERN_ACTION_DEPORTATION = _compile(
529:     r"(?<!self-)(?<!self )\b(deport(?:ed|ing|ation)?|remov(?:ed|al)|repatriat(?:ed|ion))\b"
530: )
531: ICE_OR_DHS_TARGET = (
532:     r"(?:ICE|I\.C\.E\.|Immigration and Customs Enforcement|"
533:     r"U\.S\.\s+Immigration\s+and\s+Customs\s+Enforcement|DHS|"
534:     r"Department of Homeland Security|Homeland Security)"
535: )
536: ICE_DIRECT_TARGET = (
537:     r"(?:ICE|I\.C\.E\.|Immigration and Customs Enforcement|"
538:     r"U\.S\.\s+Immigration\s+and\s+Customs\s+Enforcement)"
539: )
540: ICE_REPORT_SUBJECT = (
541:     r"(?:illegal\s+(?:alien|immigrant)s?|criminal\s+aliens?|"
542:     r"undocumented\s+(?:immigrant|alien)s?|immigration\s+(?:crime|violation)s?|"
543:     r"alien(?:s|['\u2019]s)?|immigrants?|migrants?)"
544: )
545: ICE_REPORT_PHONE = r"(?:866[-\s]?DHS[-\s]?2[-\s]?ICE|866[-\s]?347[-\s]?2423)"
546: PATTERN_ACTION_REPORT_TO_ICE = _compile(
547:     rf"\b(?:"
548:     rf"(?:call|contact)\s+(?:{ICE_DIRECT_TARGET}|{ICE_REPORT_PHONE})\b"
549:     rf"(?=.{{0,180}}\b{ICE_REPORT_SUBJECT}\b)"
550:     rf"|(?:report|tip|submit|send|notify)\b.{{0,80}}\b{ICE_REPORT_SUBJECT}\b"
551:     rf".{{0,80}}\b(?:to|at|with)\s+(?:{ICE_OR_DHS_TARGET}|{ICE_REPORT_PHONE})\b"
552:     rf"|(?:submit|send)\s+(?:a\s+)?tip\b.{{0,80}}\b(?:to|with)\s+"
553:     rf"(?:{ICE_OR_DHS_TARGET}|{ICE_REPORT_PHONE})\b"
554:     rf"(?=.{{0,120}}\b{ICE_REPORT_SUBJECT}\b)"
555:     rf"|{ICE_REPORT_SUBJECT}\b.{{0,240}}\b(?:call|contact)\s+"
556:     rf"(?:(?:our|the)\s+)?(?:{ICE_DIRECT_TARGET}|ICE\s+)?"
557:     rf"(?:tip\s*line|hotline|{ICE_REPORT_PHONE})\b"
558:     rf")"
559: )
```

scripts/tag_lexical.py:1518-1591

```
1518:     # layer); needs-vision means the frame/scene itself has not been analyzed
1519:     # (resolved only by an actual visual description, never by OCR).
1520:     if needs_ocr:
1521:         add("media-status:needs-ocr", tentative=True, source="media-metadata")
1522:
1523:     # Concatenate OCR and media-description text so a poster's stamped
1524:     # slogan or manually reviewed image description earns the same tags
1525:     # as if the words had been typed into the tweet body.
1526:     body_parts = [part for part in (text, ocr_text, transcript_text, media_text) if part]
1527:     body = " || ".join(body_parts)
1528:
1529:     if not body:
1530:         # Even with no text we still get format: + agency: + sticky
1531:         # default. Skip the regex pass.
1532:         _ensure_intrinsic_parent_topics(entries, "", add)
1533:         _maybe_immigration_default(entries, account_category, "", add)
1534:         return entries
1535:     text = body
1536:
1537:     # Single-shot regex tags
1538:     for pat, tag in (
1539:         (PATTERN_FRAME_CRIMINAL, "theme:criminal"),
1540:         (PATTERN_ACTION_DETENTION, "action:detention"),
1541:         (PATTERN_ACTION_SELF_DEPORTATION, "action:self-deportation"),
1542:         (PATTERN_ACTION_DEPORTATION, "action:deportation"),
1543:         (PATTERN_ACTION_REPORT_TO_ICE, "action:report-immigrants"),
1544:         (PATTERN_SUBJECT_CBP_HOME_APP, "subject:cbp-home-app"),
1545:         (PATTERN_SUBJECT_CELEBRITY, "subject:celebrity"),
1546:         (PATTERN_LEGAL_CRIMINAL_PROSECUTION, "legal:criminal-prosecution"),
1547:         (PATTERN_LEGAL_CIVIL_LAWSUIT, "legal:civil-lawsuit"),
1548:         (PATTERN_TOPIC_ECONOMY, "topic:economy"),
1549:         (PATTERN_TOPIC_MILITARY, "topic:military"),
```

```
1550:         (PATTERN_TOPIC_LAUDATORY, "topic:laudatory"),
1551:         (PATTERN_EVENT_PALESTINE, "event:palestine"),
1552:         (PATTERN_THEME_BORDER, "policy:border"),
1553:         (PATTERN_THEME_SANCTUARY, "policy:sanctuary-cities"),
1554:         (PATTERN_THEME_WORKSITE, "policy:worksite-enforcement"),
1555:         (PATTERN_THEME_HOMELAND, "theme:homeland"),
1556:         (PATTERN_THEME_NATIVISM, "theme:nativism"),
1557:         (PATTERN_THEME_CHRISTIANITY, "religion:christianity"),
1558:         (PATTERN_THEME_TRANSGENDER, "theme:transgender"),
1559:         (PATTERN_THEME_CIVIL_DISTURBANCE, "theme:civil-disturbance"),
1560:         (PATTERN_THEME_CBP_HOME, "policy:cbp-home"),
1561:         (PATTERN_THEME_POP_CULTURE_REFERENCE, "theme:pop-culture-reference"),
1562:         (PATTERN_LEGAL_BIRTHRIGHT_CITIZENSHIP, "legal:birthright-citizenship"),
1563:         (PATTERN_THEME_NATIVISM_BIRTHRIGHT, "theme:nativism"),
1564:         (PATTERN_SLOGAN_NICE, "slogan:nice"),
1565:         (PATTERN_SLOGAN_WORST, "slogan:worst"),
1566:         (PATTERN_SLOGAN_REPORTRECON, "slogan:reportrecon"),
1567:         (PATTERN_SLOGAN_CRIMINAL_ILLEGAL_ALIEN, "slogan:criminal-illegal-alien"),
1568:         (PATTERN_SLOGAN_ILLEGAL_ALIEN, "slogan:illegal-alien"),
1569:         (PATTERN_SLOGAN_FREE_TICKET_HOME, "slogan:free-ticket-home"),
1570:         (PATTERN_SLOGAN_GO_HOME, "slogan:go-home"),
1571:         (PATTERN_SLOGAN_PROJECT_HOMECOMING, "slogan:project-homecoming"),
1572:         (PATTERN_SLOGAN_MAGA, "slogan:maga"),
1573:         (PATTERN_SLOGAN_MAHA, "slogan:maha"),
1574:         (PATTERN_SLOGAN_MASA, "slogan:masa"),
1575:         (PATTERN_SLOGAN_AMERICA_FIRST, "slogan:america-first"),
1576:         (PATTERN_SLOGAN_GOLDEN_AGE, "slogan:golden-age"),
1577:         (PATTERN_SLOGAN_SAVE_AMERICA, "slogan:save-america"),
1578:         (PATTERN_SLOGAN_LAW_AND_ORDER, "slogan:law-and-order"),
1579:         (PATTERN_SLOGAN_PEACE_THROUGH_STRENGTH, "slogan:peace-through-strength"),
1580:         (PATTERN_SLOGAN_PROMISES_KEPT, "slogan:promises-kept"),
1581:         (PATTERN_SLOGAN_MASS_DEPORTATION, "slogan:mass-deportation"),
1582:         (PATTERN_SLOGAN_MOST_SECURE_BORDER, "slogan:most-secure-border"),
1583:         (PATTERN_SLOGAN_CATCH_RELEASE, "slogan:catch-release"),
1584:         (PATTERN_SLOGAN_FIND_AND_KILL, "slogan:find-and-kill"),
1585:         (PATTERN_SLOGAN_IMPORT_THIRD_WORLD, "slogan:import-third-world"),
1586:         (PATTERN_PHRASE_MIGRANT, "phrase:migrant"),
1587:         (PATTERN_PHRASE_IMMIGRANT, "phrase:immigrant"),
1588:         (PATTERN_THEME_STATISTICS, "format:statistics"),
1589:         (PATTERN_THEME_DIRECTIVE, "theme:directive"),
1590:         (PATTERN_ANGEL_FAMILY, "subject:angel-family"),
1591:         (PATTERN_NATIVE_BORN_CITIZEN, "subject:native-born-citizen"),
```

artist

The artist namespace is produced only from written or transcribed text. The code does not identify music by listening to audio. It matches artist names or listed signature song phrases in the combined text string.

Tag	Count	Precise plain-English rule
`artist:lee-greenwood`	4 (0.0%)	Matches Lee Greenwood, God Bless the USA, or Proud to be an American in written/transcribed text.
`artist:kid-rock`	2 (0.0%)	Matches Kid Rock in written/transcribed text.

Code text

scripts/tag_lexical.py:939-959

```
 939: # — right now artist names are almost entirely in the (un-OCR'd) images / audio,
 940: # not the tweet bodies.
 941: ARTIST_GAZETTEER: tuple[tuple[re.Pattern[str], str], ...] = (
 942:     (_compile(r"\bsydney\s+sweeney\b"), "sydney-sweeney"),
 943:     (
 944:         _compile(
 945:             r"\blee\s+greenwood\b|\bgod\s+bless\s+the\s+u\.?s\.?a\.?\b"
 946:             r"|\bproud\s+to\s+be\s+an\s+american\b"
 947:         ),
 948:         "lee-greenwood",
 949:     ),
 950:     (_compile(r"\bkid\s+rock\b"), "kid-rock"),
 951:     (_compile(r"\bvillage\s+people\b|\bmacho\s+man\b"), "village-people"),
 952:     (_compile(r"\bjason\s+aldean\b|\btry\s+that\s+in\s+a\s+small\s+town\b"), "jason-aldean"),
 953:     (_compile(r"\boliver\s+anthony\b|\brich\s+men\s+north\s+of\s+richmond\b"), "oliver-anthony"),
 954:     (_compile(r"\btaylor\s+swift\b"), "taylor-swift"),
 955:     (_compile(r"\bbeyonc[eé]\b"), "beyonce"),
 956: )
 957:
 958: # --- video:<kind> ----------------------------------------------------
 959: #
```

scripts/tag_lexical.py:1672-1675

```
1672:     # artist:<name> — named cultural figures / musicians (and signature songs).
1673:     for artist_pat, artist_slug in ARTIST_GAZETTEER:
1674:         if m := artist_pat.search(text):
1675:             add(f"artist:{artist_slug}", span=m.span())
```

country

The country namespace is produced by validated extraction. The code first captures a one- or two-word place after one of these prepositions: from, in, to, of, with, by. It then resolves that capture against the country vocabulary. It also adds country tags for curated demonyms and curated foreign city names.

Tag	Count	Precise plain-English rule
country:Mexico	771 (7.7%)	Emitted as country:Mexico when a validated candidate resolves to Mexico: a country name after from/in/to/of/with/by, a curated demonym for Mexico, or a curated foreign city mapped to Mexico.
country:El-Salvador	266 (2.7%)	Emitted as country:El-Salvador when a validated candidate resolves to El Salvador: a country name after from/in/to/of/with/by, a curated demonym for El Salvador, or a curated foreign city mapped to El Salvador.
country:Guatemala	189 (1.9%)	Emitted as country:Guatemala when a validated candidate resolves to Guatemala: a country name after from/in/to/of/with/by, a curated demonym for Guatemala, or a curated foreign city mapped to Guatemala.
country:Honduras	180 (1.8%)	Emitted as country:Honduras when a validated candidate resolves to Honduras: a country name after from/in/to/of/with/by, a curated demonym for Honduras, or a curated foreign city mapped to Honduras.
country:Iran	133 (1.3%)	Emitted as country:Iran when a validated candidate resolves to Iran: a country name after from/in/to/of/with/by, a curated demonym for Iran, or a curated foreign city mapped to Iran.
country:Venezuela	133 (1.3%)	Emitted as country:Venezuela when a validated candidate resolves to Venezuela: a country name after from/in/to/of/with/by, a curated demonym for Venezuela, or a curated foreign city mapped to Venezuela.
country:Cuba	100 (1.0%)	Emitted as country:Cuba when a validated candidate resolves to Cuba: a country name after from/in/to/of/with/by, a curated demonym for Cuba, or a curated foreign city mapped to Cuba.
country:China	98 (1.0%)	Emitted as country:China when a validated candidate resolves to China: a country name after from/in/to/of/with/by, a curated demonym for China, or a curated foreign city mapped to China.
country:Dominican-Republic	70 (0.7%)	Emitted as country:Dominican-Republic when a validated candidate resolves to Dominican Republic: a country name after from/in/to/of/with/by, a curated demonym for Dominican Republic, or a curated foreign city mapped to Dominican Republic.
country:Colombia	54 (0.5%)	Emitted as country:Colombia when a validated candidate resolves to Colombia: a country name after from/in/to/of/with/by, a curated demonym for Colombia, or a curated foreign city mapped to Colombia.
country:Laos	51 (0.5%)	Emitted as country:Laos when a validated candidate resolves to Laos: a country name after from/in/to/of/with/by, a curated demonym for Laos, or a curated foreign city mapped to Laos.
country:Ecuador	49 (0.5%)	Emitted as country:Ecuador when a validated candidate resolves to Ecuador: a country name after from/in/to/of/with/by, a curated demonym for Ecuador, or a curated foreign city mapped to Ecuador.
country:Israel	43 (0.4%)	Emitted as country:Israel when a validated candidate resolves to Israel: a country name after from/in/to/of/with/by, a curated demonym for Israel, or a curated foreign city mapped to Israel.
country:Vietnam	39 (0.4%)	Emitted as country:Vietnam when a validated candidate resolves to Vietnam: a country name after from/in/to/of/with/by, a curated demonym for Vietnam, or a curated foreign city mapped to Vietnam.
country:India	36 (0.4%)	Emitted as country:India when a validated candidate resolves to India: a country name after from/in/to/of/with/by, a curated demonym for India, or a curated foreign city mapped to India.
country:Somalia	35 (0.3%)	Emitted as country:Somalia when a validated candidate resolves to Somalia: a country name after from/in/to/of/with/by, a curated demonym for Somalia, or a curated foreign city mapped to Somalia.
country:Haiti	33 (0.3%)	Emitted as country:Haiti when a validated candidate resolves to Haiti: a country name after from/in/to/of/with/by, a curated demonym for Haiti, or a curated foreign city mapped to Haiti.
country:Brazil	31 (0.3%)	Emitted as country:Brazil when a validated candidate resolves to Brazil: a country name after from/in/to/of/with/by, a curated demonym for Brazil, or a curated foreign city mapped to Brazil.
country:Canada	31 (0.3%)	Emitted as country:Canada when a validated candidate resolves to Canada: a country name after from/in/to/of/with/by, a curated demonym for Canada, or a curated foreign city mapped to Canada.
country:Afghanistan	27 (0.3%)	Emitted as country:Afghanistan when a validated candidate resolves to Afghanistan: a country name after from/in/to/of/with/by, a curated demonym for Afghanistan, or a curated foreign city mapped to Afghanistan.
country:Jamaica	25 (0.2%)	Emitted as country:Jamaica when a validated candidate resolves to Jamaica: a country name after from/in/to/of/with/by, a curated demonym for Jamaica, or a curated foreign city mapped to Jamaica.
country:Russia	25 (0.2%)	Emitted as country:Russia when a validated candidate resolves to Russia: a country name after from/in/to/of/with/by, a curated demonym for Russia, or a curated foreign city mapped to Russia.
country:Nicaragua	22 (0.2%)	Emitted as country:Nicaragua when a validated candidate resolves to Nicaragua: a country name after from/in/to/of/with/by, a curated demonym for Nicaragua, or a curated foreign city mapped to Nicaragua.
country:Pakistan	15 (0.1%)	Emitted as country:Pakistan when a validated candidate resolves to Pakistan: a country name after from/in/to/of/with/by, a curated demonym for Pakistan, or a curated foreign city mapped to Pakistan.
country:Ukraine	15 (0.1%)	Emitted as country:Ukraine when a validated candidate resolves to Ukraine: a country name after from/in/to/of/with/by, a curated demonym for Ukraine, or a curated foreign city mapped to Ukraine.
country:Georgia	14 (0.1%)	Emitted as country:Georgia when a validated candidate resolves to Georgia: a country name after from/in/to/of/with/by, a curated demonym for Georgia, or a curated foreign city mapped to Georgia.
country:Nigeria	13 (0.1%)	Emitted as country:Nigeria when a validated candidate resolves to Nigeria: a country name after from/in/to/of/with/by, a curated demonym for Nigeria, or a curated foreign city mapped to Nigeria.
country:Argentina	11 (0.1%)	Emitted as country:Argentina when a validated candidate resolves to Argentina: a country name after from/in/to/of/with/by, a curated demonym for Argentina, or a curated foreign city mapped to Argentina.
country:Peru	11 (0.1%)	Emitted as country:Peru when a validated candidate resolves to Peru: a country name after from/in/to/of/with/by, a curated demonym for Peru, or a curated foreign city mapped to Peru.
country:Lebanon	10 (0.1%)	Emitted as country:Lebanon when a validated candidate resolves to Lebanon: a country name after from/in/to/of/with/by, a curated demonym for Lebanon, or a curated foreign city mapped to Lebanon.
country:United-States	10 (0.1%)	Emitted as country:United-States when a validated candidate resolves to United States: a country name after from/in/to/of/with/by, a curated demonym for United States, or a curated foreign city mapped to United States.

Tag	Count	Precise plain-English rule
country:Egypt	8 (0.1%)	Emitted as country:Egypt when a validated candidate resolves to Egypt: a country name after from/in/to/of/with/by, a curated demonym for Egypt, or a curated foreign city mapped to Egypt.
country:Iraq	8 (0.1%)	Emitted as country:Iraq when a validated candidate resolves to Iraq: a country name after from/in/to/of/with/by, a curated demonym for Iraq, or a curated foreign city mapped to Iraq.
country:Panama	8 (0.1%)	Emitted as country:Panama when a validated candidate resolves to Panama: a country name after from/in/to/of/with/by, a curated demonym for Panama, or a curated foreign city mapped to Panama.
country:Cambodia	7 (0.1%)	Emitted as country:Cambodia when a validated candidate resolves to Cambodia: a country name after from/in/to/of/with/by, a curated demonym for Cambodia, or a curated foreign city mapped to Cambodia.
country:Sierra-Leone	7 (0.1%)	Emitted as country:Sierra-Leone when a validated candidate resolves to Sierra Leone: a country name after from/in/to/of/with/by, a curated demonym for Sierra Leone, or a curated foreign city mapped to Sierra Leone.
country:South-Korea	7 (0.1%)	Emitted as country:South-Korea when a validated candidate resolves to South Korea: a country name after from/in/to/of/with/by, a curated demonym for South Korea, or a curated foreign city mapped to South Korea.
country:France	6 (0.1%)	Emitted as country:France when a validated candidate resolves to France: a country name after from/in/to/of/with/by, a curated demonym for France, or a curated foreign city mapped to France.
country:Guyana	6 (0.1%)	Emitted as country:Guyana when a validated candidate resolves to Guyana: a country name after from/in/to/of/with/by, a curated demonym for Guyana, or a curated foreign city mapped to Guyana.
country:Jordan	6 (0.1%)	Emitted as country:Jordan when a validated candidate resolves to Jordan: a country name after from/in/to/of/with/by, a curated demonym for Jordan, or a curated foreign city mapped to Jordan.
country:Sudan	6 (0.1%)	Emitted as country:Sudan when a validated candidate resolves to Sudan: a country name after from/in/to/of/with/by, a curated demonym for Sudan, or a curated foreign city mapped to Sudan.
country:Syria	6 (0.1%)	Emitted as country:Syria when a validated candidate resolves to Syria: a country name after from/in/to/of/with/by, a curated demonym for Syria, or a curated foreign city mapped to Syria.
country:Ghana	5 (0.0%)	Emitted as country:Ghana when a validated candidate resolves to Ghana: a country name after from/in/to/of/with/by, a curated demonym for Ghana, or a curated foreign city mapped to Ghana.
country:Japan	5 (0.0%)	Emitted as country:Japan when a validated candidate resolves to Japan: a country name after from/in/to/of/with/by, a curated demonym for Japan, or a curated foreign city mapped to Japan.
country:Myanmar	5 (0.0%)	Emitted as country:Myanmar when a validated candidate resolves to Myanmar: a country name after from/in/to/of/with/by, a curated demonym for Myanmar, or a curated foreign city mapped to Myanmar.
country:Turkey	5 (0.0%)	Emitted as country:Turkey when a validated candidate resolves to Turkey: a country name after from/in/to/of/with/by, a curated demonym for Turkey, or a curated foreign city mapped to Turkey.
country:Barbados	4 (0.0%)	Emitted as country:Barbados when a validated candidate resolves to Barbados: a country name after from/in/to/of/with/by, a curated demonym for Barbados, or a curated foreign city mapped to Barbados.
country:Cameroon	4 (0.0%)	Emitted as country:Cameroon when a validated candidate resolves to Cameroon: a country name after from/in/to/of/with/by, a curated demonym for Cameroon, or a curated foreign city mapped to Cameroon.
country:Chile	4 (0.0%)	Emitted as country:Chile when a validated candidate resolves to Chile: a country name after from/in/to/of/with/by, a curated demonym for Chile, or a curated foreign city mapped to Chile.
country:Ethiopia	4 (0.0%)	Emitted as country:Ethiopia when a validated candidate resolves to Ethiopia: a country name after from/in/to/of/with/by, a curated demonym for Ethiopia, or a curated foreign city mapped to Ethiopia.
country:Germany	4 (0.0%)	Emitted as country:Germany when a validated candidate resolves to Germany: a country name after from/in/to/of/with/by, a curated demonym for Germany, or a curated foreign city mapped to Germany.
country:Greece	4 (0.0%)	Emitted as country:Greece when a validated candidate resolves to Greece: a country name after from/in/to/of/with/by, a curated demonym for Greece, or a curated foreign city mapped to Greece.
country:Ireland	4 (0.0%)	Emitted as country:Ireland when a validated candidate resolves to Ireland: a country name after from/in/to/of/with/by, a curated demonym for Ireland, or a curated foreign city mapped to Ireland.
country:Kenya	4 (0.0%)	Emitted as country:Kenya when a validated candidate resolves to Kenya: a country name after from/in/to/of/with/by, a curated demonym for Kenya, or a curated foreign city mapped to Kenya.
country:Liberia	4 (0.0%)	Emitted as country:Liberia when a validated candidate resolves to Liberia: a country name after from/in/to/of/with/by, a curated demonym for Liberia, or a curated foreign city mapped to Liberia.
country:Philippines	4 (0.0%)	Emitted as country:Philippines when a validated candidate resolves to Philippines: a country name after from/in/to/of/with/by, a curated demonym for Philippines, or a curated foreign city mapped to Philippines.
country:Qatar	4 (0.0%)	Emitted as country:Qatar when a validated candidate resolves to Qatar: a country name after from/in/to/of/with/by, a curated demonym for Qatar, or a curated foreign city mapped to Qatar.
country:Rwanda	4 (0.0%)	Emitted as country:Rwanda when a validated candidate resolves to Rwanda: a country name after from/in/to/of/with/by, a curated demonym for Rwanda, or a curated foreign city mapped to Rwanda.
country:South-Africa	4 (0.0%)	Emitted as country:South-Africa when a validated candidate resolves to South Africa: a country name after from/in/to/of/with/by, a curated demonym for South Africa, or a curated foreign city mapped to South Africa.
country:Trinidad	4 (0.0%)	Emitted as country:Trinidad when a validated candidate resolves to Trinidad: a country name after from/in/to/of/with/by, a curated demonym for Trinidad, or a curated foreign city mapped to Trinidad.
country:Australia	3 (0.0%)	Emitted as country:Australia when a validated candidate resolves to Australia: a country name after from/in/to/of/with/by, a curated demonym for Australia, or a curated foreign city mapped to Australia.
country:Bahrain	3 (0.0%)	Emitted as country:Bahrain when a validated candidate resolves to Bahrain: a country name after from/in/to/of/with/by, a curated demonym for Bahrain, or a curated foreign city mapped to Bahrain.

Tag	Count	Precise plain-English rule
country:Bangladesh	3 (0.0%)	Emitted as country:Bangladesh when a validated candidate resolves to Bangladesh: a country name after from/in/to/of/with/by, a curated demonym for Bangladesh, or a curated foreign city mapped to Bangladesh.
country:Costa-Rica	3 (0.0%)	Emitted as country:Costa-Rica when a validated candidate resolves to Costa Rica: a country name after from/in/to/of/with/by, a curated demonym for Costa Rica, or a curated foreign city mapped to Costa Rica.
country:Italy	3 (0.0%)	Emitted as country:Italy when a validated candidate resolves to Italy: a country name after from/in/to/of/with/by, a curated demonym for Italy, or a curated foreign city mapped to Italy.
country:Senegal	3 (0.0%)	Emitted as country:Senegal when a validated candidate resolves to Senegal: a country name after from/in/to/of/with/by, a curated demonym for Senegal, or a curated foreign city mapped to Senegal.
country:Spain	3 (0.0%)	Emitted as country:Spain when a validated candidate resolves to Spain: a country name after from/in/to/of/with/by, a curated demonym for Spain, or a curated foreign city mapped to Spain.
country:Thailand	3 (0.0%)	Emitted as country:Thailand when a validated candidate resolves to Thailand: a country name after from/in/to/of/with/by, a curated demonym for Thailand, or a curated foreign city mapped to Thailand.
country:Uganda	3 (0.0%)	Emitted as country:Uganda when a validated candidate resolves to Uganda: a country name after from/in/to/of/with/by, a curated demonym for Uganda, or a curated foreign city mapped to Uganda.
country:Uzbekistan	3 (0.0%)	Emitted as country:Uzbekistan when a validated candidate resolves to Uzbekistan: a country name after from/in/to/of/with/by, a curated demonym for Uzbekistan, or a curated foreign city mapped to Uzbekistan.
country:Angola	2 (0.0%)	Emitted as country:Angola when a validated candidate resolves to Angola: a country name after from/in/to/of/with/by, a curated demonym for Angola, or a curated foreign city mapped to Angola.
country:Armenia	2 (0.0%)	Emitted as country:Armenia when a validated candidate resolves to Armenia: a country name after from/in/to/of/with/by, a curated demonym for Armenia, or a curated foreign city mapped to Armenia.
country:Azerbaijan	2 (0.0%)	Emitted as country:Azerbaijan when a validated candidate resolves to Azerbaijan: a country name after from/in/to/of/with/by, a curated demonym for Azerbaijan, or a curated foreign city mapped to Azerbaijan.
country:Burkina-Faso	2 (0.0%)	Emitted as country:Burkina-Faso when a validated candidate resolves to Burkina Faso: a country name after from/in/to/of/with/by, a curated demonym for Burkina Faso, or a curated foreign city mapped to Burkina Faso.
country:Finland	2 (0.0%)	Emitted as country:Finland when a validated candidate resolves to Finland: a country name after from/in/to/of/with/by, a curated demonym for Finland, or a curated foreign city mapped to Finland.
country:Gabon	2 (0.0%)	Emitted as country:Gabon when a validated candidate resolves to Gabon: a country name after from/in/to/of/with/by, a curated demonym for Gabon, or a curated foreign city mapped to Gabon.
country:Kazakhstan	2 (0.0%)	Emitted as country:Kazakhstan when a validated candidate resolves to Kazakhstan: a country name after from/in/to/of/with/by, a curated demonym for Kazakhstan, or a curated foreign city mapped to Kazakhstan.
country:Kyrgyzstan	2 (0.0%)	Emitted as country:Kyrgyzstan when a validated candidate resolves to Kyrgyzstan: a country name after from/in/to/of/with/by, a curated demonym for Kyrgyzstan, or a curated foreign city mapped to Kyrgyzstan.
country:Portugal	2 (0.0%)	Emitted as country:Portugal when a validated candidate resolves to Portugal: a country name after from/in/to/of/with/by, a curated demonym for Portugal, or a curated foreign city mapped to Portugal.
country:Romania	2 (0.0%)	Emitted as country:Romania when a validated candidate resolves to Romania: a country name after from/in/to/of/with/by, a curated demonym for Romania, or a curated foreign city mapped to Romania.
country:South-Sudan	2 (0.0%)	Emitted as country:South-Sudan when a validated candidate resolves to South Sudan: a country name after from/in/to/of/with/by, a curated demonym for South Sudan, or a curated foreign city mapped to South Sudan.
country:United-Kingdom	2 (0.0%)	Emitted as country:United-Kingdom when a validated candidate resolves to United Kingdom: a country name after from/in/to/of/with/by, a curated demonym for United Kingdom, or a curated foreign city mapped to United Kingdom.
country:Uruguay	2 (0.0%)	Emitted as country:Uruguay when a validated candidate resolves to Uruguay: a country name after from/in/to/of/with/by, a curated demonym for Uruguay, or a curated foreign city mapped to Uruguay.
country:Yemen	2 (0.0%)	Emitted as country:Yemen when a validated candidate resolves to Yemen: a country name after from/in/to/of/with/by, a curated demonym for Yemen, or a curated foreign city mapped to Yemen.
country:Algeria	1 (0.0%)	Emitted as country:Algeria when a validated candidate resolves to Algeria: a country name after from/in/to/of/with/by, a curated demonym for Algeria, or a curated foreign city mapped to Algeria.
country:Belarus	1 (0.0%)	Emitted as country:Belarus when a validated candidate resolves to Belarus: a country name after from/in/to/of/with/by, a curated demonym for Belarus, or a curated foreign city mapped to Belarus.
country:Belgium	1 (0.0%)	Emitted as country:Belgium when a validated candidate resolves to Belgium: a country name after from/in/to/of/with/by, a curated demonym for Belgium, or a curated foreign city mapped to Belgium.
country:Belize	1 (0.0%)	Emitted as country:Belize when a validated candidate resolves to Belize: a country name after from/in/to/of/with/by, a curated demonym for Belize, or a curated foreign city mapped to Belize.
country:Botswana	1 (0.0%)	Emitted as country:Botswana when a validated candidate resolves to Botswana: a country name after from/in/to/of/with/by, a curated demonym for Botswana, or a curated foreign city mapped to Botswana.
country:Chad	1 (0.0%)	Emitted as country:Chad when a validated candidate resolves to Chad: a country name after from/in/to/of/with/by, a curated demonym for Chad, or a curated foreign city mapped to Chad.
country:Congo	1 (0.0%)	Emitted as country:Congo when a validated candidate resolves to Congo: a country name after from/in/to/of/with/by, a curated demonym for Congo, or a curated foreign city mapped to Congo.
country:Djibouti	1 (0.0%)	Emitted as country:Djibouti when a validated candidate resolves to Djibouti: a country name after from/in/to/of/with/by, a curated demonym for Djibouti, or a curated foreign city mapped to Djibouti.
country:Dominica	1 (0.0%)	Emitted as country:Dominica when a validated candidate resolves to Dominica: a country name after from/in/to/of/with/by, a curated demonym for Dominica, or a curated foreign city mapped to Dominica.

Tag	Count	Precise plain-English rule
country:Iceland	1 (0.0%)	Emitted as country:Iceland when a validated candidate resolves to Iceland: a country name after from/in/to/of/with/by, a curated demonym for Iceland, or a curated foreign city mapped to Iceland.
country:Latvia	1 (0.0%)	Emitted as country:Latvia when a validated candidate resolves to Latvia: a country name after from/in/to/of/with/by, a curated demonym for Latvia, or a curated foreign city mapped to Latvia.
country:Mauritania	1 (0.0%)	Emitted as country:Mauritania when a validated candidate resolves to Mauritania: a country name after from/in/to/of/with/by, a curated demonym for Mauritania, or a curated foreign city mapped to Mauritania.
country:Mauritius	1 (0.0%)	Emitted as country:Mauritius when a validated candidate resolves to Mauritius: a country name after from/in/to/of/with/by, a curated demonym for Mauritius, or a curated foreign city mapped to Mauritius.
country:Montenegro	1 (0.0%)	Emitted as country:Montenegro when a validated candidate resolves to Montenegro: a country name after from/in/to/of/with/by, a curated demonym for Montenegro, or a curated foreign city mapped to Montenegro.
country:Namibia	1 (0.0%)	Emitted as country:Namibia when a validated candidate resolves to Namibia: a country name after from/in/to/of/with/by, a curated demonym for Namibia, or a curated foreign city mapped to Namibia.
country:Poland	1 (0.0%)	Emitted as country:Poland when a validated candidate resolves to Poland: a country name after from/in/to/of/with/by, a curated demonym for Poland, or a curated foreign city mapped to Poland.
country:Togo	1 (0.0%)	Emitted as country:Togo when a validated candidate resolves to Togo: a country name after from/in/to/of/with/by, a curated demonym for Togo, or a curated foreign city mapped to Togo.
country:Tonga	1 (0.0%)	Emitted as country:Tonga when a validated candidate resolves to Tonga: a country name after from/in/to/of/with/by, a curated demonym for Tonga, or a curated foreign city mapped to Tonga.
country:United-Arab-Emirates	1 (0.0%)	Emitted as country:United-Arab-Emirates when a validated candidate resolves to United Arab Emirates: a country name after from/in/to/of/with/by, a curated demonym for United Arab Emirates, or a curated foreign city mapped to United Arab Emirates.

Code text

scripts/tag_lexical.py:1217-1316

```
1217: # second word excludes those same prepositions via lookahead, so a greedy
1218: # capture can't swallow the preposition that introduces the *next* place
1219: # ("from USA to Mexico" -> "USA" + "Mexico", not "USA to"). `_resolve_vocab`
1220: # then validates against the vocab and falls back to the leading word for any
1221: # other trailing token. Kept conservative because raw country names
1222: # false-positive easily ("Chad", "Georgia").
1223: _PREPOSITIONS = "from|in|to|of|with|by"
1224: _PLACE = rf"[A-Za-z][a-zA-Z]+(?:\s+(?!(?i:{_PREPOSITIONS})\b)[A-Za-z][a-zA-Z]+)?"
1225: # "from <Country>," — anchors the ORIGIN validator.
1226: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"\b(?i:from)\s+({_PLACE})\s*,")
1227: # Any sovereign-state mention after a preposition.
1228: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"\b(?i:{_PREPOSITIONS})\s+({_PLACE})\b")
1229: # "<Place>, <State>" — anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b{_PLACE},\s+({_PLACE})\b")
1231:
1232: # Demonyms and major foreign cities -> country. The validators above need a
1233: # preposition + the exact country name, so they miss "Iranian regime" (demonym)
1234: # and "in Tehran" (city) — the most common way countries actually appear. These
1235: # maps recover them; demonyms are distinctive enough to match without a
1236: # preposition. Curated for precision: food/ambiguous demonyms ("French",
1237: # "German", "Indian", "Korean") are intentionally omitted.
1238: DEMONYM_TO_COUNTRY: dict[str, str] = {
1239:     "iranian": "Iran",
1240:     "mexican": "Mexico",
1241:     "venezuelan": "Venezuela",
1242:     "chinese": "China",
1243:     "russian": "Russia",
1244:     "cuban": "Cuba",
1245:     "colombian": "Colombia",
1246:     "salvadoran": "El Salvador",
1247:     "salvadorian": "El Salvador",
1248:     "honduran": "Honduras",
1249:     "guatemalan": "Guatemala",
1250:     "haitian": "Haiti",
1251:     "somali": "Somalia",
1252:     "somalian": "Somalia",
1253:     "afghan": "Afghanistan",
1254:     "syrian": "Syria",
1255:     "nigerian": "Nigeria",
1256:     "pakistani": "Pakistan",
1257:     "ukrainian": "Ukraine",
1258:     "israeli": "Israel",
1259:     "iraqi": "Iraq",
1260:     "egyptian": "Egypt",
1261:     "yemeni": "Yemen",
1262:     "brazilian": "Brazil",
1263:     "nicaraguan": "Nicaragua",
1264:     "ecuadorian": "Ecuador",
1265:     "peruvian": "Peru",
1266:     "jamaican": "Jamaica",
1267:     "filipino": "Philippines",
1268:     "vietnamese": "Vietnam",
1269:     "lebanese": "Lebanon",
1270:     "libyan": "Libya",
1271:     "sudanese": "Sudan",
1272:     "ethiopian": "Ethiopia",
1273:     "kenyan": "Kenya",
1274:     "moroccan": "Morocco",
1275:     "algerian": "Algeria",
1276:     "bangladeshi": "Bangladesh",
1277:     "cambodian": "Cambodia",
1278:     "indonesian": "Indonesia",
1279:     "dominican": "Dominican Republic",
1280:     "panamanian": "Panama",
1281:     "bolivian": "Bolivia",
1282:     "chilean": "Chile",
1283:     "argentine": "Argentina",
```

```
1284:     "argentinian": "Argentina",
1285:     "saudi": "Saudi Arabia",
1286:     "turkish": "Turkey",
1287: }
1288: FOREIGN_CITY_TO_COUNTRY: dict[str, str] = {
1289:     "tehran": "Iran",
1290:     "caracas": "Venezuela",
1291:     "beijing": "China",
1292:     "moscow": "Russia",
1293:     "havana": "Cuba",
1294:     "kabul": "Afghanistan",
1295:     "damascus": "Syria",
1296:     "baghdad": "Iraq",
1297:     "tripoli": "Libya",
1298:     "mogadishu": "Somalia",
1299:     "bogota": "Colombia",
1300:     "managua": "Nicaragua",
1301:     "tegucigalpa": "Honduras",
1302:     "kyiv": "Ukraine",
1303: }
1304:
1305:
1306: def _alternation_pattern(keys: list[str]) -> re.Pattern[str]:
1307:     alts = sorted((re.escape(k) for k in keys), key=len, reverse=True)
1308:     return re.compile(r"\b(" + "|".join(alts) + r")\b", re.IGNORECASE)
1309:
1310:
1311: PATTERN_DEMONYM = _alternation_pattern(list(DEMONYM_TO_COUNTRY))
1312: PATTERN_FOREIGN_CITY = _alternation_pattern(list(FOREIGN_CITY_TO_COUNTRY))
1313: # A demonym that immediately precedes one of these nouns describes a person's
1314: # nationality, so it also earns origin:<country>, not just a contextual country:.
1315: PATTERN_DEMONYM_PERSON = _compile(
1316:     r"^\W*(?:national|nationals|citizen|citizens|immigrant|immigrants|migrant|migrants|"
```

scripts/tag_lexical.py:1683-1701

```
1683:     # country:<NAME> — broader: any contextual country mention.
1684:     for m in PATTERN_COUNTRY_CANDIDATE.finditer(text):
1685:         resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1686:         if resolved:
1687:             add(f"country:{_normalize_country(resolved)}", span=m.span(1))
1688:
1689:     # Demonyms ("Iranian regime") and major foreign cities ("in Tehran") ->
1690:     # country, which the bare-name validators above can't see. A demonym
1691:     # followed by a person noun also earns origin:.
1692:     for m in PATTERN_DEMONYM.finditer(text):
1693:         country = DEMONYM_TO_COUNTRY.get(m.group(1).lower())
1694:         if country and country.lower() in COUNTRY_LOWER:
1695:             add(f"country:{_normalize_country(country)}", span=m.span(1))
1696:             if PATTERN_DEMONYM_PERSON.search(text[m.end() : m.end() + 24]):
1697:                 add(f"origin:{_normalize_country(country)}", span=m.span(1))
1698:     for m in PATTERN_FOREIGN_CITY.finditer(text):
1699:         country = FOREIGN_CITY_TO_COUNTRY.get(m.group(1).lower())
1700:         if country and country.lower() in COUNTRY_LOWER:
1701:             add(f"country:{_normalize_country(country)}", span=m.span(1))
```

scripts/tag_lexical.py:2046-2066

```
2046:     if low in ("usa", "united states"):
2047:         return "United-States"
2048:     if low == "uk":
2049:         return "United-Kingdom"
2050:     if low == "uae":
2051:         return "United-Arab-Emirates"
2052:     if low == "burma":
2053:         return "Myanmar"
2054:     return s.replace(" ", "-")
2055:
2056:
2057: def _normalize_state(s: str) -> str:
2058:     return s.replace(" ", "-")
2059:
2060:
2061: def _resolve_vocab(candidate: str, vocab: dict[str, str]) -> str | None:
2062:     """Resolve a candidate (any case, possibly two words) to its canonical
2063:     vocab spelling.
2064:
2065:     Tries the full phrase first, then its leading word, so a greedy
2066:     case-insensitive capture that swallowed a trailing lowercase token
```

crime

The crime namespace is produced by iterating CRIME_VOCAB and CRIME_SUBTYPE_VOCAB. Each regex match emits one crime tag. Some matches also emit parent tags through CRIME_PARENT_TAGS, such as rape -> sexual, murder -> homicide, fentanyl -> narcotics.

Tag	Count	Precise plain-English rule
crime:assault	911 (9.1%)	Matches the assault entry in CRIME_VOCAB/CRIME_SUBTYPE_VOCAB. If this tag has a configured parent in CRIME_PARENT_TAGS, the parent is emitted from the same text span.
crime:homicide	737 (7.4%)	Matches the homicide entry in CRIME_VOCAB/CRIME_SUBTYPE_VOCAB. If this tag has a configured parent in CRIME_PARENT_TAGS, the parent is emitted from the same text span.
crime:sexual	619 (6.2%)	Matches the sexual entry in CRIME_VOCAB/CRIME_SUBTYPE_VOCAB. If this tag has a configured parent in CRIME_PARENT_TAGS, the parent is emitted from the same text span.
crime:murder	614 (6.1%)	Matches the murder entry in CRIME_VOCAB/CRIME_SUBTYPE_VOCAB. If this tag has a configured parent in CRIME_PARENT_TAGS, the parent is emitted from the same text span.
crime:narcotics	516 (5.2%)	Matches the narcotics entry in CRIME_VOCAB/CRIME_SUBTYPE_VOCAB. If this tag has a configured parent in CRIME_PARENT_TAGS, the parent is emitted from the same text span.

Tag	Count	Precise plain-English rule
crime:trafficking	405 (4.0%)	Matches the trafficking entry in CRIME_VOCAB/CRIME_SUBTYPE_VOCAB. If this tag has a configured parent in CRIME_PARENT_TAGS, the parent is emitted from the same text span.
crime:weapon	377 (3.8%)	Matches the weapon entry in CRIME_VOCAB/CRIME_SUBTYPE_VOCAB. If this tag has a configured parent in CRIME_PARENT_TAGS, the parent is emitted from the same text span.
crime:gang	337 (3.4%)	Matches the gang entry in CRIME_VOCAB/CRIME_SUBTYPE_VOCAB. If this tag has a configured parent in CRIME_PARENT_TAGS, the parent is emitted from the same text span.
crime:rape	276 (2.8%)	Matches the rape entry in CRIME_VOCAB/CRIME_SUBTYPE_VOCAB. If this tag has a configured parent in CRIME_PARENT_TAGS, the parent is emitted from the same text span.
crime:child	256 (2.6%)	Matches the child entry in CRIME_VOCAB/CRIME_SUBTYPE_VOCAB. If this tag has a configured parent in CRIME_PARENT_TAGS, the parent is emitted from the same text span.
crime:fraud	226 (2.3%)	Matches the fraud entry in CRIME_VOCAB/CRIME_SUBTYPE_VOCAB. If this tag has a configured parent in CRIME_PARENT_TAGS, the parent is emitted from the same text span.
crime:cocaine	222 (2.2%)	Matches the cocaine entry in CRIME_VOCAB/CRIME_SUBTYPE_VOCAB. If this tag has a configured parent in CRIME_PARENT_TAGS, the parent is emitted from the same text span.
crime:child-sexual	215 (2.1%)	Matches the child-sexual entry in CRIME_VOCAB/CRIME_SUBTYPE_VOCAB. If this tag has a configured parent in CRIME_PARENT_TAGS, the parent is emitted from the same text span.
crime:battery	187 (1.9%)	Matches the battery entry in CRIME_VOCAB/CRIME_SUBTYPE_VOCAB. If this tag has a configured parent in CRIME_PARENT_TAGS, the parent is emitted from the same text span.
crime:firearm	177 (1.8%)	Matches the firearm entry in CRIME_VOCAB/CRIME_SUBTYPE_VOCAB. If this tag has a configured parent in CRIME_PARENT_TAGS, the parent is emitted from the same text span.
crime:theft	166 (1.7%)	Matches the theft entry in CRIME_VOCAB/CRIME_SUBTYPE_VOCAB. If this tag has a configured parent in CRIME_PARENT_TAGS, the parent is emitted from the same text span.
crime:robbery	164 (1.6%)	Matches the robbery entry in CRIME_VOCAB/CRIME_SUBTYPE_VOCAB. If this tag has a configured parent in CRIME_PARENT_TAGS, the parent is emitted from the same text span.
crime:fentanyl	156 (1.6%)	Matches the fentanyl entry in CRIME_VOCAB/CRIME_SUBTYPE_VOCAB. If this tag has a configured parent in CRIME_PARENT_TAGS, the parent is emitted from the same text span.
crime:ms13	156 (1.6%)	Matches the ms13 entry in CRIME_VOCAB/CRIME_SUBTYPE_VOCAB. If this tag has a configured parent in CRIME_PARENT_TAGS, the parent is emitted from the same text span.
crime:dui	148 (1.5%)	Matches the dui entry in CRIME_VOCAB/CRIME_SUBTYPE_VOCAB. If this tag has a configured parent in CRIME_PARENT_TAGS, the parent is emitted from the same text span.
crime:burglary	132 (1.3%)	Matches the burglary entry in CRIME_VOCAB/CRIME_SUBTYPE_VOCAB. If this tag has a configured parent in CRIME_PARENT_TAGS, the parent is emitted from the same text span.
crime:meth	132 (1.3%)	Matches the meth entry in CRIME_VOCAB/CRIME_SUBTYPE_VOCAB. If this tag has a configured parent in CRIME_PARENT_TAGS, the parent is emitted from the same text span.
crime:kidnap	123 (1.2%)	Matches the kidnap entry in CRIME_VOCAB/CRIME_SUBTYPE_VOCAB. If this tag has a configured parent in CRIME_PARENT_TAGS, the parent is emitted from the same text span.
crime:tren-de-aragua	117 (1.2%)	Matches the tren-de-aragua entry in CRIME_VOCAB/CRIME_SUBTYPE_VOCAB. If this tag has a configured parent in CRIME_PARENT_TAGS, the parent is emitted from the same text span.
crime:arson	29 (0.3%)	Matches the arson entry in CRIME_VOCAB/CRIME_SUBTYPE_VOCAB. If this tag has a configured parent in CRIME_PARENT_TAGS, the parent is emitted from the same text span.
crime:sodomy	23 (0.2%)	Matches the sodomy entry in CRIME_VOCAB/CRIME_SUBTYPE_VOCAB. If this tag has a configured parent in CRIME_PARENT_TAGS, the parent is emitted from the same text span.
crime:disobedience	8 (0.1%)	Matches the disobedience entry in CRIME_VOCAB/CRIME_SUBTYPE_VOCAB. If this tag has a configured parent in CRIME_PARENT_TAGS, the parent is emitted from the same text span.
crime:perjury	1 (0.0%)	Matches the perjury entry in CRIME_VOCAB/CRIME_SUBTYPE_VOCAB. If this tag has a configured parent in CRIME_PARENT_TAGS, the parent is emitted from the same text span.

Code text

scripts/tag_lexical.py:288-331

```
288: # Vocabulary for `crime:<TYPE>`. Each entry is (slug, pattern). The slug
289: # becomes the tag suffix (`crime:assault`, `crime:fentanyl`, etc.).
290: CRIME_VOCAB: tuple[tuple[str, str], ...] = (
291:     ("rape", r"\brap(?:e|ed|ing|ist)\b"),
292:     ("sodomy", r"\bsodom(?:y|ize|ized)\b"),
293:     ("homicide", r"\bhomicides?\b"),
294:     ("burglary", r"\bburglar(?:y|ize|ized|ies)\b"),
295:     ("theft", r"\btheft\b|\bsteal(?:ing)?\b|\bstole\b"),
296:     ("robbery", r"\brobber(?:y|ies)\b|\brobbed\b"),
297:     ("assault", r"\bassault(?:ed|ing|s)?\b"),
298:     ("battery", r"\bbatter(?:y|ies|ed)\b"),
299:     ("fentanyl", r"\bfentanyl\b"),
300:     ("cocaine", r"\bcocaine\b"),
301:     ("meth", r"\bmethamphetamine\b|\bmeth\b"),
302:     ("trafficking", r"\btraffick(?:ing|er|ers)\b"),
303:     ("dui", r"\bDUI\b|\bDWI\b"),
304:     ("kidnap", r"\bkidnap(?:ped|ping|per)?\b"),
305:     ("sexual", r"\b(?:sexual\s+assault|sex\s+crime|sex\s+offen[sc]e|sex\s+abuse)\b"),
306:     (
307:         "child-sexual",
308:         r"\b(?:child\s+(?:sex(?:ual)?|pornography|molest(?:ation)?)|"
309:         r"child\s+sexual|minor\s+sex(?:ual)?|sex(?:ual)?\s+abuse\s+of\s+(?:a\s+)?minor)\b",
310:     ),
311:     ("child", r"\bchild (?:abuse|sex|pornography|molest)|child sexual\b"),
```

```
312:     ("gang", r"\bgang (?:member|members|affiliation)\b|\bgang-related\b"),
313:     ("ms13", r"\bMS-?13\b"),
314:     ("tren-de-aragua", r"\bTren de Aragua\b|\bTdA\b"),
315:     ("narcotics", r"\bnarcotic[s]?\b"),
316:     ("fraud", r"\bfraud(?:ulent|ster)?\b"),
317:     (
318:         "disobedience",
319:         r"\b(?:contempt\s+of\s+court|violat(?:e|ed|ing|ion)\s+(?:of\s+)?(?:a\s+)?court\s+order|disobey(?:ed|ing)?\s+(?:a\s+)?court\s+order)\b",
320:     ),
321:     ("perjury", r"\bperjur(?:y|ed)\b|\blying\s+under\s+oath\b"),
322:     ("arson", r"\barson(?:ist)?\b"),
323:     ("weapon", r"\bweapon[s]?\b|\billegal firearm[s]?\b"),
324:     ("firearm", r"\bfirearm[s]?\b"),
325: )
326:
327: CRIME_SUBTYPE_VOCAB: tuple[tuple[str, str], ...] = (("murder", r"\bmurder(?:s|ed|ers?|ing)?\b"),)
328:
329: CRIME_PARENT_TAGS: dict[str, tuple[str, ...]] = {
330:     "crime:murder": ("crime:homicide",),
331:     "crime:rape": ("crime:sexual",),
```

scripts/tag_lexical.py:1638-1653

```
1638:     # crime:<TYPE> — every distinct match emits one tag entry.
1639:     for slug, pat_str in CRIME_VOCAB:
1640:         for m in re.finditer(pat_str, text, re.I):
1641:             add(f"crime:{slug}", span=m.span())
1642:             for parent in CRIME_PARENT_TAGS.get(f"crime:{slug}", ()):
1643:                 add(parent, span=m.span())
1644:     for slug, pat_str in CRIME_SUBTYPE_VOCAB:
1645:         for m in re.finditer(pat_str, text, re.I):
1646:             tag = f"crime:{slug}"
1647:             add(tag, span=m.span())
1648:             for parent in CRIME_PARENT_TAGS.get(tag, ()):
1649:                 add(parent, span=m.span())
1650:
1651:     if m := _martyrdom_match(text, account_category, entries):
1652:         add("theme:martyrdom", span=m.span())
1653:
```

event

The event namespace is produced by direct event regexes and disturbance-cluster regexes. Disturbance clusters require the city name and unrest/federal-response language within 180 characters of each other.

Tag	Count	Precise plain-English rule
`event:palestine`	36 (0.4%)	Matches Palestine/Palestinian(s), Gaza/Gazan(s), Hamas, Israel-Hamas, Israel-Palestine/Palestinian, or West Bank.
`event:los-angeles-disturbance`	15 (0.1%)	Matches the los angeles disturbance disturbance cluster: the city name must occur within 180 characters of riot/unrest/protest/federal-response language, in either order.
`event:portland-disturbance`	6 (0.1%)	Matches the portland disturbance disturbance cluster: the city name must occur within 180 characters of riot/unrest/protest/federal-response language, in either order.
`event:minneapolis-disturbance`	5 (0.0%)	Matches the minneapolis disturbance disturbance cluster: the city name must occur within 180 characters of riot/unrest/protest/federal-response language, in either order.

Code text

scripts/tag_lexical.py:639-644

```
639: PATTERN_EVENT_PALESTINE = _compile(
640:     r"\b(?:palestin(?:e|ian)s?|gaza(?:n)?s?|hamas|israel[- ]hamas|"
641:     r"israel[- ]palestin(?:e|ian)|west\s+bank)\b"
642: )
643: GENERAL_TOPIC_PATTERNS: tuple[tuple[str, re.Pattern[str]], ...] = (
644:     ("immigration", _compile(r"\b(immigration|migrant|border|illegal alien|asylum)\b")),
```

scripts/tag_lexical.py:727-763

```
727:     r"violent\s+protests?|anti[- ]ICE\s+(?:riot(?:s|ers|ing)?|protests?)|"
728:     r"violent\s+demonstrators?|street\s+violence|mob\s+violence)\b"
729: )
730: DISTURBANCE_CITY_PATTERNS: tuple[tuple[str, re.Pattern[str]], ...] = (
731:     (
732:         "los-angeles-disturbance",
733:         _compile(
734:             r"\b(?:Los\s+Angeles|L\.A\.|LA)\b.{0,180}\b(?:riot(?:s|ers|ing)?|"
735:             r"civil\s+disturbance|civil\s+unrest|violent\s+protests?|anti[- ]ICE\s+"
736:             r"(?:riot(?:s|ers|ing)?|protests?)|federal\s+building|concrete\s+at\s+DHS\s+agents)\b"
737:             r"|\b(?:riot(?:s|ers|ing)?|civil\s+disturbance|civil\s+unrest|violent\s+protests?|"
738:             r"anti[- ]ICE\s+(?:riot(?:s|ers|ing)?|protests?)|federal\s+building|"
739:             r"concrete\s+at\s+DHS\s+agents)\b.{0,180}\b(?:Los\s+Angeles|L\.A\.|LA)\b"
740:         ),
741:     ),
742:     (
743:         "minneapolis-disturbance",
744:         _compile(
745:             r"\bMinneapolis\b.{0,180}\b(?:Bovino|riot(?:s|ers|ing)?|civil\s+disturbance|"
746:             r"civil\s+unrest|violent\s+protests?|protests?|tear\s+gas|pellet\s+guns?|"
747:             r"heavy[- ]handed|lawsuits?|killing|killed|disaster)\b"
748:             r"|\b(?:Bovino|riot(?:s|ers|ing)?|civil\s+disturbance|civil\s+unrest|"
749:             r"violent\s+protests?|protests?|tear\s+gas|pellet\s+guns?|heavy[- ]handed|"
750:             r"lawsuits?|killing|killed|disaster)\b.{0,180}\bMinneapolis\b"
751:         ),
752:     ),
753:     (
754:         "portland-disturbance",
755:         _compile(
756:             r"\bPortland\b.{0,180}\b(?:riot(?:s|ers|ing)?|civil\s+disturbance|civil\s+unrest|"
```

```
757:             r"violent\s+protests?|anti[- ]ICE\s+(?:riot(?:s|ers|ing)?|protests?)|"
758:             r"ICE\s+(?:detention|facility)|Antifa|domestic\s+terrorism)\b"
759:             r"|\b(?:riot(?:s|ers|ing)?|civil\s+disturbance|civil\s+unrest|violent\s+protests?|"
760:             r"anti[- ]ICE\s+(?:riot(?:s|ers|ing)?|protests?)|ICE\s+(?:detention|facility)|"
761:             r"Antifa|domestic\s+terrorism)\b.{0,180}\bPortland\b"
762:         ),
763:     ),
```

scripts/tag_lexical.py:1609-1612

```
1609:     if m := _coded_nativism_match(text, entries):
1610:         add("theme:nativism", span=m.span())
1611:
1612:     if m := _theme_religion_match(text):
```

genre

The genre namespace is produced from produced-video regexes, parody franchise matches, media-description sidecar tags, and the lineup composite. The lineup composite is exact: reply + theme:criminal + exactly one attached media item.

Tag	Count	Precise plain-English rule
genre:lineup	670 (6.7%)	Composite rule: tweet_type is reply, theme:criminal already fired, and media_count equals exactly 1.
genre:advertisement	439 (4.4%)	On video posts or sidecar text, matches new ad/advert/spot/commercial, ad/advertisement/commercial/promo/promotional video/campaign spot, campaign/recruitment ad/spot/video/campaign, join ICE/CBP/HSI language, join.ice.gov, apply today/now, Promises Made/Kept, or most secure border in American history.
genre:psa	128 (1.3%)	On video posts or sidecar text, matches PSA/public service announcement, learn/find out/more info/visit website at, did you know, know your rights/facts/signs, protect yourself, stay safe, report a/suspicious, or call 1-800/888/877/866 numbers.
genre:recruitment	126 (1.3%)	On video posts or sidecar text, matches join.ice.gov, DHS/ICE/CBP/HSI career/join/job/recruiting URLs, hiring/recruiting, recruitment/hiring ad/spot/video/campaign, join ICE/CBP/HSI/DHS/team/family, apply today/now, answer the call, or serve your/our/their/his/her country/nation/community.
genre:utopian	44 (0.4%)	Matches utopian, idealized, aspirational, golden age, bright future, morning in America, or sunlit/heroic/triumphal within 80 characters of montage/vision/future/aesthetic.
genre:music-video	36 (0.4%)	On video posts, matches explicit music-video wording: official music video, set to music/the song/the track, official video/audio for, or lyric/lyrics video. Speech/press-conference indicators suppress this tag.
genre:dystopian	18 (0.2%)	Matches dystopian, sci-fi/science fiction, cyberpunk, or dark/bleak/apocalyptic/hellscape/surveillance-state/futuristic within 80 characters of future/city/vision/scene/aesthetic.
genre:parody	5 (0.0%)	Emitted when any franchise parody pattern fires; it is not emitted by itself.
genre:war-movie	2 (0.0%)	Matches war movie/war film/action movie, or cinematic/movie-trailer/trailer-style/dramatic within 80 characters of war/battle/combat/military/raid/operation in either order.

Code text

scripts/tag_lexical.py:841-936

```
841: )
842:
843: # --- genre:parody + parody:<franchise> ----------------------------------
844: # A political/spoof parody of a recognizable media franchise.
845: # Franchise gazetteer: distinctive phrase -> franchise slug.
846: # "this is the way" alone is too generic (e.g. Mandalorian is pop-culture but
847: # appears in non-parody political text too), so we require at least ONE other
848: # Star Wars cue in the same text, OR the canonical "May the 4th/fourth be with
849: # you" phrase which is uniquely Star Wars in context.
850: PARODY_FRANCHISE_GAZETTEER: tuple[tuple[re.Pattern[str], str], ...] = (
851:     (
852:         _compile(
853:             r"\bmay\s+the\s+(?:4th|fourth)\s+be\s+with\s+you\b"
854:             r"|\bthe\s+force\s+(?:is|be|will\s+be)\s+with\b"
855:             r"|\b(?:jedi|sith|death\s+star|lightsaber|darth\s+vader|skywalker)\b"
856:             r"|\b(?:galaxy|republic|empire|rebellion|mandalorian)\b.{0,200}\b(?:jedi|sith|death\s+star|the\s+way)\b"
857:         ),
858:         "star-wars",
859:     ),
860:     (
861:         _compile(
862:             r"\bman\s+of\s+steel\b|\bkrypton(?:ite)?\b|\bfortress\s+of\s+solitude\b"
863:             r"|\bup,?\s+up,?\s+and\s+away\b|\bsuperman\b.{0,80}\b(?:cape|hero|krypton|steel|villain)\b"
864:         ),
865:         "superman",
866:     ),
867:     (
868:         _compile(
869:             r"\btop\s+gun\b|\bthe\s+need\s+for\s+speed\b|\bi\s+feel\s+the\s+need\b"
870:             r"|\bdanger\s+zone\b.{0,80}\b(?:maverick|jet|fly|top\s+gun)\b"
871:         ),
872:         "top-gun",
873:     ),
874:     (
875:         _compile(
876:             r"\bwinter\s+is\s+coming\b|\bgame\s+of\s+thrones\b|\bthe\s+iron\s+throne\b"
877:             r"|\byou\s+know\s+nothing,?\s+jon\s+snow\b"
878:         ),
879:         "game-of-thrones",
880:     ),
881:     (
882:         _compile(
```

```
            r"\bone\s+ring\s+to\s+rule\s+them\s+all\b|\byou\s+shall\s+not\s+pass\b"
            r"|\bmy\s+precious\b|\b(?:mordor|gandalf|frodo|the\s+shire)\b"
        ),
        "lord-of-the-rings",
    ),
    (
        _compile(
            r"\bhasta\s+la\s+vista,?\s+baby\b|\bskynet\b|\bthe\s+terminator\b"
            r"|\bi'?ll\s+be\s+back\b.{0,80}\b(?:terminator|machine|robot|cyborg)\b"
        ),
        "terminator",
    ),
    (
        _compile(r"\beye\s+of\s+the\s+tiger\b|\brocky\s+balboa\b|\bgonna\s+fly\s+now\b"),
        "rocky",
    ),
    (
        _compile(
            r"\bshaken,?\s+not\s+stirred\b|\blicen[sc]e\s+to\s+kill\b|\bagent\s+007\b|\bjames\s+bond\b"
        ),
        "james-bond",
    ),
    (
        _compile(
            r"\bavengers\s+assemble\b|\bwakanda\s+forever\b|\binfinity\s+(?:gauntlet|stones?)\b"
            r"|\bi\s+am\s+iron\s+man\b|\bthanos\b"
        ),
        "marvel",
    ),
    (
        _compile(r"\bwho\s+(?:you|ya)\s+gonna\s+call\b|\bghostbusters\b"),
        "ghostbusters",
    ),
    (
        _compile(r"\boffer\s+(?:he|you|they)\s+can't\s+refuse\b|\bthe\s+godfather\b"),
        "godfather",
    ),
    (
        _compile(r"\bpawn\s+stars\b"),
        "pawn-stars",
    ),
)
# Emit genre:parody ONLY when a franchise match fires; it never fires alone.
# A broader multi-cue Star Wars check: text must contain "this is the way"
# AND at least one other signature Star Wars cue.
PATTERN_PARODY_STAR_WARS_MULTI = _compile(r"\bthis\s+is\s+the\s+way\b")
PATTERN_PARODY_STAR_WARS_EXTRA_CUE = _compile(
    r"\b(?:may\s+the\s+(?:4th|fourth)\s+be\s+with\s+you|the\s+force|jedi|sith|death\s+star|"
    r"a\s+galaxy|galaxy\s+(?:far|that)|lightsaber|darth|yoda|skywalker|mandalorian)\b"
)

# --- artist:<name> -----------------------------------------------------
# Named cultural figures / musicians and their signature songs. Identifying an
# artist from audio is impossible (the track can't be heard), so these fire only
```

scripts/tag_lexical.py:1055-1089

```
PRODUCED_VIDEO_GENRE_PATTERNS: tuple[tuple[str, re.Pattern[str]], ...] = (
    ("genre:music-video", PATTERN_VIDEO_MUSIC_VIDEO),
    ("genre:psa", PATTERN_VIDEO_PSA),
    ("genre:recruitment", PATTERN_VIDEO_RECRUITMENT),
    ("genre:advertisement", PATTERN_VIDEO_AD),
    (
        "genre:war-movie",
        _compile(
            r"\bwar[- ]movie\b|\bwar[- ]film\b|\baction[- ]movie\b"
            r"|\b(?:cinematic|movie[- ]trailer|trailer[- ]style|dramatic)\b.{0,80}\b(?:war|battle|combat|military|raid|operation)\b"
            r"|\b(?:war|battle|combat|military|raid|operation)\b.{0,80}\b(?:cinematic|movie[- ]trailer|trailer[- ]style|dramatic)\b"
        ),
    ),
    (
        "genre:utopian",
        _compile(
            r"\butopian\b|\bidealized\b|\baspirational\b|\b(?:golden\s+age|bright\s+future|morning\s+in\s+america)\b"
            r"|\b(?:sunlit|heroic|triumphal)\b.{0,80}\b(?:montage|vision|future|aesthetic)\b"
        ),
    ),
    (
        "genre:dystopian",
        _compile(
            r"\bdystopian\b|\bsci[- ]?fi\b|\bscience[- ]fiction\b|\bcyberpunk\b"
            r"|\b(?:dark|bleak|apocalyptic|hellscape|surveillance[- ]state|futuristic)\b.{0,80}\b(?:future|city|vision|scene|aesthetic)\b"
        ),
    ),
)
PRODUCED_VIDEO_STRUCTURE_TAGS = {
    "video:produced",
    "genre:music-video",
    "video:montage",
    "video:text-overlay",
    "video:voiceover",
}
```

scripts/tag_lexical.py:1709-1715

```
    # genre:lineup: composite replies that hit theme:criminal with one photo.
    if (
        tweet_type == "reply"
        and any(e["tag"] == "theme:criminal" for e in entries)
        and media_count == 1
    ):
        add("genre:lineup")
```

scripts/tag_lexical.py:1730-1768

```
    #   long   : > 120s
    # We tag the bucket regardless of whether a kind matched so the viewer
    # can filter "all videos under 30s" without depending on text cues.
    if video_count > 0:
        for pat, tag in (
```

```
1735:             (PATTERN_VIDEO_BODYCAM, "video:bodycam"),
1736:             (PATTERN_VIDEO_INTERVIEW, "video:interview"),
1737:             (PATTERN_VIDEO_NEWS_CLIP, "video:news-clip"),
1738:             (PATTERN_VIDEO_SPEECH, "video:speech"),
1739:         ):
1740:             m = pat.search(text)
1741:             if m:
1742:                 add(tag, span=m.span())
1743:         if PATTERN_AUDIO_MUSIC_CONTEXT.search(text):
1744:             add("audio:music-likely")
1745:         if PATTERN_AUDIO_MUSIC_CONTEXT.search(reply_context_text):
1746:             add("audio:music-likely", source="reply-context")
1747:         if any(e["tag"] in {"genre:music-video"} for e in entries):
1748:             add("audio:music-likely")
1749:         if m := PATTERN_PRODUCED_VIDEO_STYLE.search(text):
1750:             add("video:produced", span=m.span())
1751:         if video_max_duration_sec is not None and video_max_duration_sec > 0:
1752:             if video_max_duration_sec <= 30:
1753:                 add("video:short")
1754:             elif video_max_duration_sec <= 120:
1755:                 add("video:medium")
1756:             else:
1757:                 add("video:long")
1758:         # Derive genre:music-video ONLY from an explicit genre:music-video
1759:         # signal (set by the conservative describe_media / manual-review rules) —
1760:         # NEVER from audio:music-likely, which is an incidental acoustic heuristic.
1761:         # Also suppress it on speech / press-conference clips.
1762:         if not speech_indicator_present and any(e["tag"] == "genre:music-video" for e in entries):
1763:             add("genre:music-video")
1764:         if any(
1765:             e["tag"] in PRODUCED_VIDEO_STRUCTURE_TAGS or str(e["tag"]).startswith("genre:")
1766:             for e in entries
1767:         ):
1768:             add("video:produced")
```

legal

The legal namespace is produced by regexes for criminal prosecution posture, civil litigation posture, and the literal phrase birthright citizenship.

Tag	Count	Precise plain-English rule
`legal:criminal-prosecution`	1,856 (18.5%)	Matches prosecution words: prosecute/prosecuted/prosecution, indict/indictment, charged with, criminal complaint/charge/case/prosecution, arraign/arraignment, plead guilty/not guilty, convict/conviction, sentence/sentencing, felony charges, or misdemeanor charges.
`legal:civil-lawsuit`	30 (0.3%)	Matches civil lawsuit/suit/action/case/complaint/litigation, lawsuit, filed a lawsuit/suit/civil action/complaint, sue/sued/sues/suing, injunction, temporary restraining order, TRO, settlement agreement, or consent decree.
`legal:birthright-citizenship`	3 (0.0%)	Matches the literal phrase birthright citizenship.

Code text

scripts/tag_lexical.py:814-826

```
 814: PATTERN_LEGAL_BIRTHRIGHT_CITIZENSHIP = _compile(r"\bbirthright\s+citizenship\b")
 815:
 816: # Nativist birthright framing: "actual/real/true birthright of (every/all)
 817: # Americans", "American('s) birthright", "our birthright" + harm verb.
 818: # This is SEPARATE from bare "birthright citizenship" (which is just a policy
 819: # term) — we want nativism only when the framing posits Americans' heritage
 820: # entitlement as being stolen/diluted/erased/destroyed.
 821: PATTERN_THEME_NATIVISM_BIRTHRIGHT = _compile(
 822:     r"\b(?:actual|real|true)\s+birthright\s+of\s+(?:every|all)?\s*americans?\b"
 823:     r"|\bamerican(?:'?s?)?\s+birthright\b"
 824:     r"|\b(?:our|the)\s+birthright\b.{0,160}\b(?:steal|steals?|stolen|dilut|eras|destroy|destroyed|replac|betray|undermin)\b"
 825:     r"|\b(?:steal|steals?|stolen|dilut|eras|destroy|destroyed|replac|betray|undermin)\b.{0,160}\b(?:our|the)\s+birthright\b"
 826:     r"|\b(?:destroy(?:ing)?|destroys?|erasing?|replac(?:e|ing)?|betray(?:ing)?|undermin(?:e|ing)?|dilut(?:e|ing)?)"
```

scripts/tag_lexical.py:1178-1199

```
1178: PATTERN_LEGAL_CRIMINAL_PROSECUTION = _compile(
1179:     r"\bprosecut(?:e|ed|ing|ion|ions)\b"
1180:     r"|\bindict(?:ed|ment|ments)?\b"
1181:     r"|\bcharged\s+with\b"
1182:     r"|\bcriminal\s+(?:complaint|charges?|case|prosecution)\b"
1183:     r"|\barraign(?:ed|ment)?\b"
1184:     r"|\bple(?:a|d|aded|ads)\s+(?:guilty|not\s+guilty)\b"
1185:     r"|\bconvict(?:ed|ion|ions)?\b"
1186:     r"|\bsentenc(?:ed|ing)\b"
1187:     r"|\bfelony\s+charges?\b"
1188:     r"|\bmisdemeanor\s+charges?\b"
1189: )
1190: PATTERN_LEGAL_CIVIL_LAWSUIT = _compile(
1191:     r"\bcivil\s+(?:lawsuit|suit|action|case|complaint|litigation)\b"
1192:     r"|\blawsuits?\b"
1193:     r"|\bfiled\s+(?:a\s+)?(?:lawsuit|suit|civil\s+action|complaint)\b"
1194:     r"|\bsu(?:e|ed|es|ing)\b"
1195:     r"|\binjunction\b"
1196:     r"|\btemporary\s+restraining\s+order\b"
1197:     r"|\bTRO\b"
1198:     r"|\bsettlement\s+agreement\b"
1199:     r"|\bconsent\s+decree\b"
```

scripts/tag_lexical.py:1546-1563

```
1546:         (PATTERN_LEGAL_CRIMINAL_PROSECUTION, "legal:criminal-prosecution"),
1547:         (PATTERN_LEGAL_CIVIL_LAWSUIT, "legal:civil-lawsuit"),
1548:         (PATTERN_TOPIC_ECONOMY, "topic:economy"),
1549:         (PATTERN_TOPIC_MILITARY, "topic:military"),
1550:         (PATTERN_TOPIC_LAUDATORY, "topic:laudatory"),
```

```
1551:         (PATTERN_EVENT_PALESTINE, "event:palestine"),
1552:         (PATTERN_THEME_BORDER, "policy:border"),
1553:         (PATTERN_THEME_SANCTUARY, "policy:sanctuary-cities"),
1554:         (PATTERN_THEME_WORKSITE, "policy:worksite-enforcement"),
1555:         (PATTERN_THEME_HOMELAND, "theme:homeland"),
1556:         (PATTERN_THEME_NATIVISM, "theme:nativism"),
1557:         (PATTERN_THEME_CHRISTIANITY, "religion:christianity"),
1558:         (PATTERN_THEME_TRANSGENDER, "theme:transgender"),
1559:         (PATTERN_THEME_CIVIL_DISTURBANCE, "theme:civil-disturbance"),
1560:         (PATTERN_THEME_CBP_HOME, "policy:cbp-home"),
1561:         (PATTERN_THEME_POP_CULTURE_REFERENCE, "theme:pop-culture-reference"),
1562:         (PATTERN_LEGAL_BIRTHRIGHT_CITIZENSHIP, "legal:birthright-citizenship"),
1563:         (PATTERN_THEME_NATIVISM_BIRTHRIGHT, "theme:nativism"),
```

military

The military namespace is produced from text regexes for service branches and from account-mention aliases. Any military branch tag also adds topic:military.

Tag	Count	Precise plain-English rule
`military:navy`	171 (1.7%)	Matches navy branch terms in MILITARY_VOCAB or a mention alias mapped to military:navy; then implies topic:military.
`military:coast-guard`	142 (1.4%)	Matches coast guard branch terms in MILITARY_VOCAB or a mention alias mapped to military:coast-guard; then implies topic:military.
`military:army`	43 (0.4%)	Matches army branch terms in MILITARY_VOCAB or a mention alias mapped to military:army; then implies topic:military.
`military:national-guard`	38 (0.4%)	Matches national guard branch terms in MILITARY_VOCAB or a mention alias mapped to military:national-guard; then implies topic:military.
`military:air-force`	37 (0.4%)	Matches air force branch terms in MILITARY_VOCAB or a mention alias mapped to military:air-force; then implies topic:military.
`military:marines`	15 (0.1%)	Matches marines branch terms in MILITARY_VOCAB or a mention alias mapped to military:marines; then implies topic:military.
`military:space-force`	5 (0.0%)	Matches space force branch terms in MILITARY_VOCAB or a mention alias mapped to military:space-force; then implies topic:military.

Code text

scripts/tag_lexical.py:344-411

```
344:     {
345:         "ICEgov",
346:         "CBP",
347:         "USCIS",
348:         "DHSgov",
349:         "WhiteHouse",
350:         "POTUS",
351:         "PressSec",
352:         "USDOL",
353:         "RapidResponse47",
354:         "HSI_HQ",
355:         "USBPChief",
356:         "SecMullinDHS",
357:         "AGPamBondi",
358:         "StateDept",
359:         "DOJgov",
360:         "FBI",
361:         "DEAHQ",
362:         "USMarshalsHQ",
363:         "HHSGov",
364:         "USTreasury",
365:         "FEMA",
366:         "DeptofWar",
367:         "CENTCOM",
368:         "Southcom",
369:         "EPA",
370:         "USCG",
371:         "USCGAcademy",
372:         "ComdtUSCG",
373:         "VComdtUSCG",
374:         "USCGLANTAREA",
375:         "USCGPACAREA",
376:         "USCGSoutheast",
377:         "USCGHeartland",
378:         "USCGNorCal",
379:         "USCG_Tri_State",
380:         "USArmyNorth",
381:         "USNationalGuard",
382:         "USNorthernCmd",
383:         "TSA",
384:         "SecretService",
385:         "CISAgov",
386:         "CISAInfraSec",
387:         "CISACyber",
388:         "FLETC",
389:         "ODNIgov",
390:         "NSAGov",
391:         "IPRCenter",
392:         "USAttyEssayli",
393:         "USAttyPirro",
394:         "ERO_LosAngeles",
395:         "ERO_HQ",
396:     }
397: )
```

```
398:
399: AGENCY_MENTION_ALIASES: dict[str, str] = {
400:     **{handle.lower(): handle for handle in AGENCY_HANDLES},
401:     "fbidirectorkash": "FBI",
402:     "fbimostwanted": "FBI",
403:     "fbimnneapolis": "FBI",
404:     "fbiminneapolis": "FBI",
405:     "fbitampa": "FBI",
406:     "thejusticedept": "DOJgov",
407:     "justiceoig": "DOJgov",
408:     "epaleezeldin": "EPA",
409:     "secwar": "DeptofWar",
410: }
411:
```

scripts/tag_lexical.py:568-619

```
568:     r"soldiers?|sailors?|airmen|marines|guardsmen|army|navy|air force|space force|"
569:     r"marine corps|coast guard|national guard|USAF|USSF|USMC|pentagon|"
570:     r"department of defense|DoD|DOD|veterans affairs|combat|battlefield|"
571:     r"war zone|deployed|deployment|carrier strike group|aircraft carrier|"
572:     r"carrier air wing|CVN\s*\d+|USS\s+[A-Z][A-Za-z0-9-]+|USNS\s+[A-Z][A-Za-z0-9-]+|"
573:     r"service academ(?:y|ies)|military academy|naval academy|coast guard academy|"
574:     r"air force academy|west point|USMA|USNA|USCGA|cadets?|midshipmen|"
575:     r"commander[- ]in[- ]chief|commandant)\b"
576: )
577: MILITARY_VOCAB: tuple[tuple[str, re.Pattern[str]], ...] = (
578:     ("army", _compile(r"\b(?:(?:u\.s\.\s+)?army|soldiers?|west\s+point|USMA)\b")),
579:     (
580:         "navy",
581:         _compile(
582:             r"\b(?:(?:u\.s\.\s+)?navy|sailors?|naval\s+academy|USNA|midshipmen|"
583:             r"carrier\s+strike\s+group|aircraft\s+carrier|carrier\s+air\s+wing|"
584:             r"CVN\s*\d+|USS\s+[A-Z][A-Za-z0-9-]+|USNS\s+[A-Z][A-Za-z0-9-]+)\b"
585:         ),
586:     ),
587:     (
588:         "air-force",
589:         _compile(r"\b(?:(?:u\.s\.\s+)?air\s+force|usaf|airm[ae]n|air\s+force\s+academy)\b"),
590:     ),
591:     ("space-force", _compile(r"\b(?:(?:u\.s\.\s+)?space\s+force|ussf)\b")),
592:     ("marines", _compile(r"\b(?:marine\s+corps|u\.s\.\s+marines?|usmc|marines)\b")),
593:     (
594:         "coast-guard",
595:         _compile(
596:             r"\b(?:coast\s+guard|USCG|USCGA|coast\s+guard\s+academy|coasties?|coast\s+guards?m[ae]n)\b"
597:         ),
598:     ),
599:     ("national-guard", _compile(r"\b(?:national\s+guard|national\s+guards?m[ae]n)\b")),
600: )
601: MILITARY_MENTION_ALIASES: dict[str, str] = {
602:     "usarmynorth": "army",
603:     "usnationalguard": "national-guard",
604:     "usguard": "national-guard",
605:     "usnavy": "navy",
606:     "usairforce": "air-force",
607:     "usspaceforce": "space-force",
608:     "usmc": "marines",
609:     "marines": "marines",
610:     "uscg": "coast-guard",
611:     "uscgacademy": "coast-guard",
612:     "comdtuscg": "coast-guard",
613:     "vcomdtuscg": "coast-guard",
614:     "uscglantarea": "coast-guard",
615:     "uscgpacarea": "coast-guard",
616:     "uscgsoutheast": "coast-guard",
617:     "uscgheartland": "coast-guard",
618:     "uscgnorcal": "coast-guard",
619:     "uscg_tri_state": "coast-guard",
```

scripts/tag_lexical.py:1631-1635

```
1631:     for slug, pat in MILITARY_VOCAB:
1632:         for m in pat.finditer(text):
1633:             add(f"military:{slug}", span=m.span())
1634:
1635:     if _general_topic_score(text) >= 3:
```

origin

The origin namespace is stricter than country. It is produced when the text says from <country>, with a comma after the country, or when a curated demonym is immediately followed by a person-origin noun such as national, citizen, immigrant, migrant, man, woman, descent, or origin.

Tag	Count	Precise plain-English rule
`origin:Mexico`	378 (3.8%)	Emitted as origin:Mexico when the text has from Mexico, with a comma after the country, or a demonym for Mexico followed immediately by a person-origin noun.
`origin:El-Salvador`	94 (0.9%)	Emitted as origin:El-Salvador when the text has from El Salvador, with a comma after the country, or a demonym for El Salvador followed immediately by a person-origin noun.
`origin:Guatemala`	91 (0.9%)	Emitted as origin:Guatemala when the text has from Guatemala, with a comma after the country, or a demonym for Guatemala followed immediately by a person-origin noun.
`origin:Honduras`	86 (0.9%)	Emitted as origin:Honduras when the text has from Honduras, with a comma after the country, or a demonym for Honduras followed immediately by a person-origin noun.
`origin:Venezuela`	54 (0.5%)	Emitted as origin:Venezuela when the text has from Venezuela, with a comma after the country, or a demonym for Venezuela followed immediately by a person-origin noun.

Tag	Count	Precise plain-English rule
origin:Cuba	52 (0.5%)	Emitted as origin:Cuba when the text has from Cuba, with a comma after the country, or a demonym for Cuba followed immediately by a person-origin noun.
origin:Ecuador	25 (0.2%)	Emitted as origin:Ecuador when the text has from Ecuador, with a comma after the country, or a demonym for Ecuador followed immediately by a person-origin noun.
origin:China	23 (0.2%)	Emitted as origin:China when the text has from China, with a comma after the country, or a demonym for China followed immediately by a person-origin noun.
origin:Iran	23 (0.2%)	Emitted as origin:Iran when the text has from Iran, with a comma after the country, or a demonym for Iran followed immediately by a person-origin noun.
origin:Laos	23 (0.2%)	Emitted as origin:Laos when the text has from Laos, with a comma after the country, or a demonym for Laos followed immediately by a person-origin noun.
origin:Colombia	22 (0.2%)	Emitted as origin:Colombia when the text has from Colombia, with a comma after the country, or a demonym for Colombia followed immediately by a person-origin noun.
origin:Vietnam	18 (0.2%)	Emitted as origin:Vietnam when the text has from Vietnam, with a comma after the country, or a demonym for Vietnam followed immediately by a person-origin noun.
origin:Haiti	16 (0.2%)	Emitted as origin:Haiti when the text has from Haiti, with a comma after the country, or a demonym for Haiti followed immediately by a person-origin noun.
origin:Afghanistan	15 (0.1%)	Emitted as origin:Afghanistan when the text has from Afghanistan, with a comma after the country, or a demonym for Afghanistan followed immediately by a person-origin noun.
origin:Dominican-Republic	14 (0.1%)	Emitted as origin:Dominican-Republic when the text has from Dominican Republic, with a comma after the country, or a demonym for Dominican Republic followed immediately by a person-origin noun.
origin:Nicaragua	14 (0.1%)	Emitted as origin:Nicaragua when the text has from Nicaragua, with a comma after the country, or a demonym for Nicaragua followed immediately by a person-origin noun.
origin:India	13 (0.1%)	Emitted as origin:India when the text has from India, with a comma after the country, or a demonym for India followed immediately by a person-origin noun.
origin:Jamaica	13 (0.1%)	Emitted as origin:Jamaica when the text has from Jamaica, with a comma after the country, or a demonym for Jamaica followed immediately by a person-origin noun.
origin:Somalia	13 (0.1%)	Emitted as origin:Somalia when the text has from Somalia, with a comma after the country, or a demonym for Somalia followed immediately by a person-origin noun.
origin:Brazil	12 (0.1%)	Emitted as origin:Brazil when the text has from Brazil, with a comma after the country, or a demonym for Brazil followed immediately by a person-origin noun.
origin:Peru	6 (0.1%)	Emitted as origin:Peru when the text has from Peru, with a comma after the country, or a demonym for Peru followed immediately by a person-origin noun.
origin:Guyana	5 (0.0%)	Emitted as origin:Guyana when the text has from Guyana, with a comma after the country, or a demonym for Guyana followed immediately by a person-origin noun.
origin:Nigeria	5 (0.0%)	Emitted as origin:Nigeria when the text has from Nigeria, with a comma after the country, or a demonym for Nigeria followed immediately by a person-origin noun.
origin:Russia	5 (0.0%)	Emitted as origin:Russia when the text has from Russia, with a comma after the country, or a demonym for Russia followed immediately by a person-origin noun.
origin:Ukraine	5 (0.0%)	Emitted as origin:Ukraine when the text has from Ukraine, with a comma after the country, or a demonym for Ukraine followed immediately by a person-origin noun.
origin:Cambodia	4 (0.0%)	Emitted as origin:Cambodia when the text has from Cambodia, with a comma after the country, or a demonym for Cambodia followed immediately by a person-origin noun.
origin:Syria	4 (0.0%)	Emitted as origin:Syria when the text has from Syria, with a comma after the country, or a demonym for Syria followed immediately by a person-origin noun.
origin:Kenya	3 (0.0%)	Emitted as origin:Kenya when the text has from Kenya, with a comma after the country, or a demonym for Kenya followed immediately by a person-origin noun.
origin:Sierra-Leone	3 (0.0%)	Emitted as origin:Sierra-Leone when the text has from Sierra Leone, with a comma after the country, or a demonym for Sierra Leone followed immediately by a person-origin noun.
origin:Sudan	3 (0.0%)	Emitted as origin:Sudan when the text has from Sudan, with a comma after the country, or a demonym for Sudan followed immediately by a person-origin noun.
origin:Angola	2 (0.0%)	Emitted as origin:Angola when the text has from Angola, with a comma after the country, or a demonym for Angola followed immediately by a person-origin noun.
origin:Barbados	2 (0.0%)	Emitted as origin:Barbados when the text has from Barbados, with a comma after the country, or a demonym for Barbados followed immediately by a person-origin noun.
origin:Canada	2 (0.0%)	Emitted as origin:Canada when the text has from Canada, with a comma after the country, or a demonym for Canada followed immediately by a person-origin noun.
origin:Chile	2 (0.0%)	Emitted as origin:Chile when the text has from Chile, with a comma after the country, or a demonym for Chile followed immediately by a person-origin noun.
origin:Egypt	2 (0.0%)	Emitted as origin:Egypt when the text has from Egypt, with a comma after the country, or a demonym for Egypt followed immediately by a person-origin noun.
origin:Ghana	2 (0.0%)	Emitted as origin:Ghana when the text has from Ghana, with a comma after the country, or a demonym for Ghana followed immediately by a person-origin noun.

Tag	Count	Precise plain-English rule
origin:Iraq	2 (0.0%)	Emitted as origin:Iraq when the text has from Iraq, with a comma after the country, or a demonym for Iraq followed immediately by a person-origin noun.
origin:Pakistan	2 (0.0%)	Emitted as origin:Pakistan when the text has from Pakistan, with a comma after the country, or a demonym for Pakistan followed immediately by a person-origin noun.
origin:Panama	2 (0.0%)	Emitted as origin:Panama when the text has from Panama, with a comma after the country, or a demonym for Panama followed immediately by a person-origin noun.
origin:Philippines	2 (0.0%)	Emitted as origin:Philippines when the text has from Philippines, with a comma after the country, or a demonym for Philippines followed immediately by a person-origin noun.
origin:Thailand	2 (0.0%)	Emitted as origin:Thailand when the text has from Thailand, with a comma after the country, or a demonym for Thailand followed immediately by a person-origin noun.
origin:Turkey	2 (0.0%)	Emitted as origin:Turkey when the text has from Turkey, with a comma after the country, or a demonym for Turkey followed immediately by a person-origin noun.
origin:Argentina	1 (0.0%)	Emitted as origin:Argentina when the text has from Argentina, with a comma after the country, or a demonym for Argentina followed immediately by a person-origin noun.
origin:Azerbaijan	1 (0.0%)	Emitted as origin:Azerbaijan when the text has from Azerbaijan, with a comma after the country, or a demonym for Azerbaijan followed immediately by a person-origin noun.
origin:Bangladesh	1 (0.0%)	Emitted as origin:Bangladesh when the text has from Bangladesh, with a comma after the country, or a demonym for Bangladesh followed immediately by a person-origin noun.
origin:Burkina-Faso	1 (0.0%)	Emitted as origin:Burkina-Faso when the text has from Burkina Faso, with a comma after the country, or a demonym for Burkina Faso followed immediately by a person-origin noun.
origin:Cameroon	1 (0.0%)	Emitted as origin:Cameroon when the text has from Cameroon, with a comma after the country, or a demonym for Cameroon followed immediately by a person-origin noun.
origin:Costa-Rica	1 (0.0%)	Emitted as origin:Costa-Rica when the text has from Costa Rica, with a comma after the country, or a demonym for Costa Rica followed immediately by a person-origin noun.
origin:Germany	1 (0.0%)	Emitted as origin:Germany when the text has from Germany, with a comma after the country, or a demonym for Germany followed immediately by a person-origin noun.
origin:Ireland	1 (0.0%)	Emitted as origin:Ireland when the text has from Ireland, with a comma after the country, or a demonym for Ireland followed immediately by a person-origin noun.
origin:Israel	1 (0.0%)	Emitted as origin:Israel when the text has from Israel, with a comma after the country, or a demonym for Israel followed immediately by a person-origin noun.
origin:Italy	1 (0.0%)	Emitted as origin:Italy when the text has from Italy, with a comma after the country, or a demonym for Italy followed immediately by a person-origin noun.
origin:Kazakhstan	1 (0.0%)	Emitted as origin:Kazakhstan when the text has from Kazakhstan, with a comma after the country, or a demonym for Kazakhstan followed immediately by a person-origin noun.
origin:Latvia	1 (0.0%)	Emitted as origin:Latvia when the text has from Latvia, with a comma after the country, or a demonym for Latvia followed immediately by a person-origin noun.
origin:Liberia	1 (0.0%)	Emitted as origin:Liberia when the text has from Liberia, with a comma after the country, or a demonym for Liberia followed immediately by a person-origin noun.
origin:Mauritania	1 (0.0%)	Emitted as origin:Mauritania when the text has from Mauritania, with a comma after the country, or a demonym for Mauritania followed immediately by a person-origin noun.
origin:Myanmar	1 (0.0%)	Emitted as origin:Myanmar when the text has from Myanmar, with a comma after the country, or a demonym for Myanmar followed immediately by a person-origin noun.
origin:Rwanda	1 (0.0%)	Emitted as origin:Rwanda when the text has from Rwanda, with a comma after the country, or a demonym for Rwanda followed immediately by a person-origin noun.
origin:Senegal	1 (0.0%)	Emitted as origin:Senegal when the text has from Senegal, with a comma after the country, or a demonym for Senegal followed immediately by a person-origin noun.
origin:United-Kingdom	1 (0.0%)	Emitted as origin:United-Kingdom when the text has from United Kingdom, with a comma after the country, or a demonym for United Kingdom followed immediately by a person-origin noun.

Code text

scripts/tag_lexical.py:1217-1230

```
1217: # second word excludes those same prepositions via lookahead, so a greedy
1218: # capture can't swallow the preposition that introduces the *next* place
1219: # ("from USA to Mexico" -> "USA" + "Mexico", not "USA to"). `_resolve_vocab`
1220: # then validates against the vocab and falls back to the leading word for any
1221: # other trailing token. Kept conservative because raw country names
1222: # false-positive easily ("Chad", "Georgia").
1223: _PREPOSITIONS = "from|in|to|of|with|by"
1224: _PLACE = rf"[A-Za-z][a-zA-Z]+(?:\s+(?!(?i:{_PREPOSITIONS})\b)[A-Za-z][a-zA-Z]+)?"
1225: # "from <Country>," — anchors the ORIGIN validator.
1226: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"\b(?i:from)\s+({_PLACE})\s*,")
1227: # Any sovereign-state mention after a preposition.
1228: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"\b(?i:{_PREPOSITIONS})\s+({_PLACE})\b")
1229: # "<Place>, <State>" — anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b{_PLACE},\s+({_PLACE})\b")
```

scripts/tag_lexical.py:1238-1316

```
1238: DEMONYM_TO_COUNTRY: dict[str, str] = {
1239:     "iranian": "Iran",
1240:     "mexican": "Mexico",
1241:     "venezuelan": "Venezuela",
```

```
    "chinese": "China",
    "russian": "Russia",
    "cuban": "Cuba",
    "colombian": "Colombia",
    "salvadoran": "El Salvador",
    "salvadorian": "El Salvador",
    "honduran": "Honduras",
    "guatemalan": "Guatemala",
    "haitian": "Haiti",
    "somali": "Somalia",
    "somalian": "Somalia",
    "afghan": "Afghanistan",
    "syrian": "Syria",
    "nigerian": "Nigeria",
    "pakistani": "Pakistan",
    "ukrainian": "Ukraine",
    "israeli": "Israel",
    "iraqi": "Iraq",
    "egyptian": "Egypt",
    "yemeni": "Yemen",
    "brazilian": "Brazil",
    "nicaraguan": "Nicaragua",
    "ecuadorian": "Ecuador",
    "peruvian": "Peru",
    "jamaican": "Jamaica",
    "filipino": "Philippines",
    "vietnamese": "Vietnam",
    "lebanese": "Lebanon",
    "libyan": "Libya",
    "sudanese": "Sudan",
    "ethiopian": "Ethiopia",
    "kenyan": "Kenya",
    "moroccan": "Morocco",
    "algerian": "Algeria",
    "bangladeshi": "Bangladesh",
    "cambodian": "Cambodia",
    "indonesian": "Indonesia",
    "dominican": "Dominican Republic",
    "panamanian": "Panama",
    "bolivian": "Bolivia",
    "chilean": "Chile",
    "argentine": "Argentina",
    "argentinian": "Argentina",
    "saudi": "Saudi Arabia",
    "turkish": "Turkey",
}
FOREIGN_CITY_TO_COUNTRY: dict[str, str] = {
    "tehran": "Iran",
    "caracas": "Venezuela",
    "beijing": "China",
    "moscow": "Russia",
    "havana": "Cuba",
    "kabul": "Afghanistan",
    "damascus": "Syria",
    "baghdad": "Iraq",
    "tripoli": "Libya",
    "mogadishu": "Somalia",
    "bogota": "Colombia",
    "managua": "Nicaragua",
    "tegucigalpa": "Honduras",
    "kyiv": "Ukraine",
}


def _alternation_pattern(keys: list[str]) -> re.Pattern[str]:
    alts = sorted((re.escape(k) for k in keys), key=len, reverse=True)
    return re.compile(r"\b(" + "|".join(alts) + r")\b", re.IGNORECASE)


PATTERN_DEMONYM = _alternation_pattern(list(DEMONYM_TO_COUNTRY))
PATTERN_FOREIGN_CITY = _alternation_pattern(list(FOREIGN_CITY_TO_COUNTRY))
# A demonym that immediately precedes one of these nouns describes a person's
# nationality, so it also earns origin:<country>, not just a contextual country:.
PATTERN_DEMONYM_PERSON = _compile(
    r"^\W*(?:national|nationals|citizen|citizens|immigrant|immigrants|migrant|migrants|"
```

scripts/tag_lexical.py:1677-1698

```
    # origin:<COUNTRY> — validated against the sovereign-state vocab.
    for m in PATTERN_ORIGIN_CANDIDATE.finditer(text):
        resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
        if resolved:
            add(f"origin:{_normalize_country(resolved)}", span=m.span(1))

    # country:<NAME> — broader: any contextual country mention.
    for m in PATTERN_COUNTRY_CANDIDATE.finditer(text):
        resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
        if resolved:
            add(f"country:{_normalize_country(resolved)}", span=m.span(1))

    # Demonyms ("Iranian regime") and major foreign cities ("in Tehran") ->
    # country, which the bare-name validators above can't see. A demonym
    # followed by a person noun also earns origin:.
    for m in PATTERN_DEMONYM.finditer(text):
        country = DEMONYM_TO_COUNTRY.get(m.group(1).lower())
        if country and country.lower() in COUNTRY_LOWER:
            add(f"country:{_normalize_country(country)}", span=m.span(1))
            if PATTERN_DEMONYM_PERSON.search(text[m.end() : m.end() + 24]):
                add(f"origin:{_normalize_country(country)}", span=m.span(1))
    for m in PATTERN_FOREIGN_CITY.finditer(text):
```

parody

The parody namespace is produced by the franchise gazetteer. A franchise match adds both genre:parody and parody:<franchise>. Star Wars also has a two-cue rule: this is the way plus at least one other Star Wars cue.

Tag	Count	Precise plain-English rule
parody:star-wars	2 (0.0%)	Matches May the 4th/fourth be with you, force-with-you language, Jedi/Sith/Death Star/lightsaber/Darth Vader/Skywalker, or galaxy/republic/empire/rebellion/Mandalorian within 200 characters of Jedi/Sith/Death Star/the way. Also matches this is the way plus another Star Wars cue.
parody:game-of-thrones	1 (0.0%)	Matches winter is coming, Game of Thrones, the iron throne, or you know nothing Jon Snow.
parody:ghostbusters	1 (0.0%)	Matches who you/ya gonna call or Ghostbusters.
parody:pawn-stars	1 (0.0%)	Matches Pawn Stars.

Code text

scripts/tag_lexical.py:841-936

```
)

# --- genre:parody + parody:<franchise> ----------------------------------
# A political/spoof parody of a recognizable media franchise.
# Franchise gazetteer: distinctive phrase -> franchise slug.
# "this is the way" alone is too generic (e.g. Mandalorian is pop-culture but
# appears in non-parody political text too), so we require at least ONE other
# Star Wars cue in the same text, OR the canonical "May the 4th/fourth be with
# you" phrase which is uniquely Star Wars in context.
PARODY_FRANCHISE_GAZETTEER: tuple[tuple[re.Pattern[str], str], ...] = (
    (
        _compile(
            r"\bmay\s+the\s+(?:4th|fourth)\s+be\s+with\s+you\b"
            r"|\bthe\s+force\s+(?:is|be|will\s+be)\s+with\b"
            r"|\b(?:jedi|sith|death\s+star|lightsaber|darth\s+vader|skywalker)\b"
            r"|\b(?:galaxy|republic|empire|rebellion|mandalorian)\b.{0,200}\b(?:jedi|sith|death\s+star|the\s+way)\b"
        ),
        "star-wars",
    ),
    (
        _compile(
            r"\bman\s+of\s+steel\b|\bkrypton(?:ite)?\b|\bfortress\s+of\s+solitude\b"
            r"|\bup,?\s+up,?\s+and\s+away\b|\bsuperman\b.{0,80}\b(?:cape|hero|krypton|steel|villain)\b"
        ),
        "superman",
    ),
    (
        _compile(
            r"\btop\s+gun\b|\bthe\s+need\s+for\s+speed\b|\bi\s+feel\s+the\s+need\b"
            r"|\bdanger\s+zone\b.{0,80}\b(?:maverick|jet|fly|top\s+gun)\b"
        ),
        "top-gun",
    ),
    (
        _compile(
            r"\bwinter\s+is\s+coming\b|\bgame\s+of\s+thrones\b|\bthe\s+iron\s+throne\b"
            r"|\byou\s+know\s+nothing,?\s+jon\s+snow\b"
        ),
        "game-of-thrones",
    ),
    (
        _compile(
            r"\bone\s+ring\s+to\s+rule\s+them\s+all\b|\byou\s+shall\s+not\s+pass\b"
            r"|\bmy\s+precious\b|\b(?:mordor|gandalf|frodo|the\s+shire)\b"
        ),
        "lord-of-the-rings",
    ),
    (
        _compile(
            r"\bhasta\s+la\s+vista,?\s+baby\b|\bskynet\b|\bthe\s+terminator\b"
            r"|\bi'?ll\s+be\s+back\b.{0,80}\b(?:terminator|machine|robot|cyborg)\b"
        ),
        "terminator",
    ),
    (
        _compile(r"\beye\s+of\s+the\s+tiger\b|\brocky\s+balboa\b|\bgonna\s+fly\s+now\b"),
        "rocky",
    ),
    (
        _compile(
            r"\bshaken,?\s+not\s+stirred\b|\blicen[sc]e\s+to\s+kill\b|\bagent\s+007\b|\bjames\s+bond\b"
        ),
        "james-bond",
    ),
    (
        _compile(
            r"\bavengers\s+assemble\b|\bwakanda\s+forever\b|\binfinity\s+(?:gauntlet|stones?)\b"
            r"|\bi\s+am\s+iron\s+man\b|\bthanos\b"
        ),
        "marvel",
    ),
    (
        _compile(r"\bwho\s+(?:you|ya)\s+gonna\s+call\b|\bghostbusters\b"),
        "ghostbusters",
    ),
    (
        _compile(r"\boffer\s+(?:he|you|they)\s+can't\s+refuse\b|\bthe\s+godfather\b"),
        "godfather",
    ),
    (
        _compile(r"\bpawn\s+stars\b"),
        "pawn-stars",
    ),
)
# Emit genre:parody ONLY when a franchise match fires; it never fires alone.
# A broader multi-cue Star Wars check: text must contain "this is the way"
# AND at least one other signature Star Wars cue.
```

```
928: PATTERN_PARODY_STAR_WARS_MULTI = _compile(r"\bthis\s+is\s+the\s+way\b")
929: PATTERN_PARODY_STAR_WARS_EXTRA_CUE = _compile(
930:     r"\b(?:may\s+the\s+(?:4th|fourth)\s+be\s+with\s+you|the\s+force|jedi|sith|death\s+star|"
931:     r"a\s+galaxy|galaxy\s+(?:far|that)|lightsaber|darth|yoda|skywalker|mandalorian)\b"
932: )
933:
934: # --- artist:<name> ----------------------------------------------------
935: # Named cultural figures / musicians and their signature songs. Identifying an
936: # artist from audio is impossible (the track can't be heard), so these fire only
```

scripts/tag_lexical.py:1656-1670

```
1656:         if m := franchise_pat.search(text):
1657:             add("genre:parody", span=m.span())
1658:             add(f"parody:{franchise_slug}", span=m.span())
1659:             add("theme:pop-culture-reference")
1660:     # Also catch "this is the way" + one more Star Wars cue (e.g. "a galaxy").
1661:     if (
1662:         PATTERN_PARODY_STAR_WARS_MULTI.search(text)
1663:         and PATTERN_PARODY_STAR_WARS_EXTRA_CUE.search(text)
1664:         and not any(e["tag"] == "parody:star-wars" for e in entries)
1665:     ):
1666:         m_way = PATTERN_PARODY_STAR_WARS_MULTI.search(text)
1667:         if m_way:
1668:             add("genre:parody", span=m_way.span())
1669:             add("parody:star-wars", span=m_way.span())
1670:             add("theme:pop-culture-reference")
```

phrase

The phrase namespace is produced by literal regexes for migrant(s) and immigrant(s). Each phrase also implies topic:immigration.

Tag	Count	Precise plain-English rule
phrase:immigrant	234 (2.3%)	Matches immigrant/immigrants.
phrase:migrant	142 (1.4%)	Matches migrant/migrants.

Code text

scripts/tag_lexical.py:1176-1177

```
1176: PATTERN_PHRASE_MIGRANT = _compile(r"\bmigrants?\b")
1177: PATTERN_PHRASE_IMMIGRANT = _compile(r"\bimmigrants?\b")
```

scripts/tag_lexical.py:1586-1587

```
1586:         (PATTERN_PHRASE_MIGRANT, "phrase:migrant"),
1587:         (PATTERN_PHRASE_IMMIGRANT, "phrase:immigrant"),
```

scripts/tag_lexical.py:1883-1905

```
1883:     "slogan:criminal-illegal-alien": ("topic:immigration",),
1884:     "slogan:free-ticket-home": ("topic:immigration",),
1885:     "slogan:go-home": ("topic:immigration",),
1886:     "slogan:illegal-alien": ("topic:immigration",),
1887:     "slogan:mass-deportation": ("topic:immigration",),
1888:     "slogan:masa": ("topic:immigration",),
1889:     "slogan:find-and-kill": ("topic:immigration",),
1890:     "slogan:import-third-world": ("topic:immigration",),
1891:     "legal:birthright-citizenship": ("topic:immigration",),
1892:     "genre:parody": ("theme:pop-culture-reference",),
1893:     "slogan:most-secure-border": ("topic:immigration", "policy:border"),
1894:     "slogan:catch-release": ("topic:immigration",),
1895:     "slogan:project-homecoming": ("topic:immigration",),
1896:     "slogan:maga": ("topic:general",),
1897:     "slogan:maha": ("topic:general",),
1898:     "slogan:america-first": ("topic:general",),
1899:     "slogan:golden-age": ("topic:general",),
1900:     "slogan:save-america": ("topic:general",),
1901:     "slogan:law-and-order": ("topic:general",),
1902:     "slogan:peace-through-strength": ("topic:general",),
1903:     "slogan:promises-kept": ("topic:general",),
1904:     "phrase:immigrant": ("topic:immigration",),
1905:     "phrase:migrant": ("topic:immigration",),
```

policy

The policy namespace is produced by regexes for named policy areas: border, sanctuary, worksite enforcement, and CBP Home/self-deportation.

Tag	Count	Precise plain-English rule
policy:border	1,071 (10.7%)	Matches border, southwest border, crossing, or border wall.
policy:cbp-home	236 (2.4%)	Matches CBP Home, Project Homecoming, self-deport variants, free/complimentary ticket/flight home language, exit bonus, take control of departure, or go home within 120 characters of alien/migrant/CBP Home/self-deport/deport context.
policy:sanctuary-cities	191 (1.9%)	Matches sanctuary city/cities, sanctuary jurisdiction, sanctuary policy/policies, or sanctuary state.
policy:worksite-enforcement	21 (0.2%)	Matches worksite, workplace, I-9, E-Verify, employer audit, or employer raid.

Code text

scripts/tag_lexical.py:652-656
```
652: PATTERN_THEME_BORDER = _compile(r"\b(border|southwest border|crossing|border wall)\b")
653: PATTERN_THEME_SANCTUARY = _compile(
654:     r"\b(sanctuary (?:cit(?:y|ies)|jurisdiction|polic(?:y|ies))|sanctuary state)\b"
655: )
656: PATTERN_THEME_WORKSITE = _compile(r"\b(worksite|workplace|I-9|E-Verify|employer (?:audit|raid))\b")
```

scripts/tag_lexical.py:786-797
```
786: PATTERN_THEME_CBP_HOME = _compile(
787:     r"\bCBP\s+Home\b"
788:     r"|\bProject\s+Homecoming\b"
789:     r"|\bself[- ]deport(?:ed|ing|ation)?\b"
790:     r"|\bfree\s+(?:flight|plane\s+ticket|ticket)\s+home\b"
791:     r"|\bcomplimentary\s+plane\s+ticket\s+home\b"
792:     r"|\bexit\s+bonus\b"
793:     r"|\btake\s+control\s+of\s+(?:your|their|his|her)\s+departure\b"
794:     r"|\b(?:illegal\s+aliens?|aliens?|migrants?|CBP\s+Home|self[- ]deport|deport)\b"
795:     r"[\s\S]{0,120}\bgo\s+home\b"
796:     r"|\bgo\s+home\b[\s\S]{0,120}\b"
797:     r"(?:illegal\s+aliens?|aliens?|migrants?|CBP\s+Home|self[- ]deport|deport)\b"
```

scripts/tag_lexical.py:1552-1560
```
1552:         (PATTERN_THEME_BORDER, "policy:border"),
1553:         (PATTERN_THEME_SANCTUARY, "policy:sanctuary-cities"),
1554:         (PATTERN_THEME_WORKSITE, "policy:worksite-enforcement"),
1555:         (PATTERN_THEME_HOMELAND, "theme:homeland"),
1556:         (PATTERN_THEME_NATIVISM, "theme:nativism"),
1557:         (PATTERN_THEME_CHRISTIANITY, "religion:christianity"),
1558:         (PATTERN_THEME_TRANSGENDER, "theme:transgender"),
1559:         (PATTERN_THEME_CIVIL_DISTURBANCE, "theme:civil-disturbance"),
1560:         (PATTERN_THEME_CBP_HOME, "policy:cbp-home"),
```

religion

The religion namespace in this workbook contains Christianity. It is produced by a Christian-specific regex over the combined text string.

Tag	Count	Precise plain-English rule
religion:christianity	85 (0.8%)	Matches Christian/Christianity/Christians, Judeo-Christian, Christian faith/values/church/heritage/nation, Jesus/Jesus Christ, Christ is king/our lord, Bible/biblical/scripture/scriptural, Bible book chapter:verse references, or in Jesus' name.

Code text

scripts/tag_lexical.py:681-697
```
681: PATTERN_THEME_CHRISTIANITY = _compile(
682:     r"\bchristian(?:ity|s)?\b"
683:     r"|\bjudeo[- ]christian\b"
684:     r"|\bchristian\s+(?:faith|values?|church|churches|heritage|nation)\b"
685:     r"|\bjesus(?:\s+christ)?\b"
686:     r"|\bchrist\s+(?:is\s+king|the\s+king|our\s+lord)\b"
687:     r"|\b(?:bible|biblical|scripture|scriptural)\b"
688:     r"|\b(?:[1-3]\s+)?(?:Genesis|Exodus|Leviticus|Numbers|Deuteronomy|Joshua|Judges|Ruth|"
689:     r"Samuel|Kings|Chronicles|Ezra|Nehemiah|Esther|Job|Psalms?|Proverbs|Ecclesiastes|"
690:     r"Song\s+of\s+Songs|Isaiah|Jeremiah|Lamentations|Ezekiel|Daniel|Hosea|Joel|Amos|"
691:     r"Obadiah|Jonah|Micah|Nahum|Habakkuk|Zephaniah|Haggai|Zechariah|Malachi|Matthew|"
692:     r"Mark|Luke|John|Acts|Romans|Corinthians|Galatians|Ephesians|Philippians|"
693:     r"Colossians|Thessalonians|Timothy|Titus|Philemon|Hebrews|James|Peter|Jude|"
694:     r"Revelation)\s+\d{1,3}:\d{1,3}(?:[-\u2013]\d{1,3})?\b"
695:     r"|\bin\s+jesus(?:'s|')?\s+name\b"
696: )
697: PATTERN_THEME_RELIGION = _compile(
```

scripts/tag_lexical.py:1557-1557
```
1557:         (PATTERN_THEME_CHRISTIANITY, "religion:christianity"),
```

slogan

The slogan namespace is produced by individual recurring-phrase regexes. Some are literal phrases; some require nearby context, such as go home within 120 characters of immigration/deportation language.

Tag	Count	Precise plain-English rule
slogan:illegal-alien	2,287 (22.9%)	Matches illegal alien or illegal aliens.
slogan:criminal-illegal-alien	1,402 (14.0%)	Matches criminal illegal alien or criminal illegal aliens.
slogan:worst	222 (2.2%)	Matches the literal phrase WORST OF THE WORST.
slogan:masa	140 (1.4%)	Matches Make America Safe Again.
slogan:america-first	72 (0.7%)	Matches America First or AmericaFirst.
slogan:maga	58 (0.6%)	Matches MAGA or Make America Great Again.
slogan:law-and-order	51 (0.5%)	Matches Law and Order or Law & Order.
slogan:promises-kept	50 (0.5%)	Matches Promises Made, Promises Kept, with comma/semicolon/colon flexibility, or Promises Kept.

Tag	Count	Precise plain-English rule
slogan:most-secure-border	45 (0.4%)	Matches most secure border or most secured border.
slogan:golden-age	42 (0.4%)	Matches Golden Age or Welcome to the Golden Age.
slogan:free-ticket-home	26 (0.3%)	Matches free ticket home, free flight home, free plane ticket home, complimentary plane ticket home, or free flight to your/their/his/her home country.
slogan:mass-deportation	26 (0.3%)	Matches Mass Deportation or Mass Deportations.
slogan:save-america	24 (0.2%)	Matches Save America or SAVEAMERICA.
slogan:go-home	16 (0.2%)	Matches go home only when it appears within 120 characters before or after immigration/deportation context: illegal alien(s), alien(s), migrant(s), CBP Home, self-deport, or deport.
slogan:peace-through-strength	12 (0.1%)	Matches Peace Through Strength.
slogan:catch-release	11 (0.1%)	Matches catch-and-release or catch and release.
slogan:import-third-world	11 (0.1%)	Matches import the Third World or if you import the Third World.
slogan:maha	11 (0.1%)	Matches MAHA or Make America Healthy Again.
slogan:nice	9 (0.1%)	Matches NICE day, NICE morning, ICE is NICE, or NICE city.
slogan:project-homecoming	4 (0.0%)	Matches Project Homecoming.
slogan:find-and-kill	2 (0.0%)	Matches we will find you and/& kill you, with optional repeated we will before kill, or find you and/& kill you.
slogan:reportrecon	2 (0.0%)	Matches Report. Recon. Raid. with flexible whitespace after the periods.

Code text

scripts/tag_lexical.py:832-839

```
PATTERN_SLOGAN_FIND_AND_KILL = _compile(
    r"\bwe\s+will\s+find\s+you\s+(?:and|&)\s+(?:we\s+will\s+)?kill\s+you\b"
    r"|\bfind\s+you\s+(?:and|&)\s+kill\s+you\b"
)

# --- slogan:import-third-world ------------------------------------------
PATTERN_SLOGAN_IMPORT_THIRD_WORLD = _compile(
    r"\bimport\s+the\s+third\s+world\b"
```

scripts/tag_lexical.py:1145-1175

```
PATTERN_SLOGAN_NICE = _compile(r"\b(NICE day|NICE morning|ICE is NICE|NICE city)\b")
PATTERN_SLOGAN_WORST = _compile(r"\bWORST OF THE WORST\b")
PATTERN_SLOGAN_REPORTRECON = _compile(r"\bReport\.\s*Recon\.\s*Raid\.")
PATTERN_SLOGAN_CRIMINAL_ILLEGAL_ALIEN = _compile(r"\bcriminal\s+illegal\s+aliens?\b")
PATTERN_SLOGAN_ILLEGAL_ALIEN = _compile(r"\billegal\s+aliens?\b")
PATTERN_SLOGAN_FREE_TICKET_HOME = _compile(
    r"\bfree\s+(?:ticket|flight|plane\s+ticket)\s+home\b"
    r"|\bcomplimentary\s+plane\s+ticket\s+home\b"
    r"|\bfree\s+flight\s+to\s+(?:your|their|his|her)\s+home\s+country\b"
)
PATTERN_SLOGAN_GO_HOME = _compile(
    r"\b(?:illegal\s+aliens?|aliens?|migrants?|CBP\s+Home|self[- ]deport|deport)\b"
    r"[\s\S]{0,120}\bgo\s+home\b"
    r"|\bgo\s+home\b[\s\S]{0,120}\b"
    r"(?:illegal\s+aliens?|aliens?|migrants?|CBP\s+Home|self[- ]deport|deport)\b"
)
PATTERN_SLOGAN_PROJECT_HOMECOMING = _compile(r"\bProject\s+Homecoming\b")
PATTERN_SLOGAN_MAGA = _compile(r"\bMAGA\b|\bMake\s+America\s+Great\s+Again\b")
PATTERN_SLOGAN_MAHA = _compile(r"\bMAHA\b|\bMake\s+America\s+Healthy\s+Again\b")
PATTERN_SLOGAN_MASA = _compile(r"\bMake\s+America\s+Safe\s+Again\b")
PATTERN_SLOGAN_AMERICA_FIRST = _compile(r"\bAmerica\s+First\b|\bAmericaFirst\b")
PATTERN_SLOGAN_GOLDEN_AGE = _compile(r"\bGolden\s+Age\b|\bWelcome\s+to\s+the\s+Golden\s+Age\b")
PATTERN_SLOGAN_SAVE_AMERICA = _compile(r"\bSave\s+America\b|\bSAVEAMERICA\b")
PATTERN_SLOGAN_LAW_AND_ORDER = _compile(r"\bLaw\s+and\s+Order\b|\bLaw\s*&\s*Order\b")
PATTERN_SLOGAN_PEACE_THROUGH_STRENGTH = _compile(r"\bPeace\s+Through\s+Strength\b")
PATTERN_SLOGAN_PROMISES_KEPT = _compile(
    r"\bPromises?\s+Made[,;:]?\s+Promises?\s+Kept\b|\bPromises?\s+Kept\b"
)
PATTERN_SLOGAN_MASS_DEPORTATION = _compile(r"\bMass\s+Deportations?\b")
PATTERN_SLOGAN_MOST_SECURE_BORDER = _compile(r"\bmost\s+secure(?:d)?\s+border\b")
PATTERN_SLOGAN_CATCH_RELEASE = _compile(r"\bcatch(?:-and-|\s+and\s+)release\b")
```

scripts/tag_lexical.py:1564-1585

```
        (PATTERN_SLOGAN_NICE, "slogan:nice"),
        (PATTERN_SLOGAN_WORST, "slogan:worst"),
        (PATTERN_SLOGAN_REPORTRECON, "slogan:reportrecon"),
        (PATTERN_SLOGAN_CRIMINAL_ILLEGAL_ALIEN, "slogan:criminal-illegal-alien"),
        (PATTERN_SLOGAN_ILLEGAL_ALIEN, "slogan:illegal-alien"),
        (PATTERN_SLOGAN_FREE_TICKET_HOME, "slogan:free-ticket-home"),
        (PATTERN_SLOGAN_GO_HOME, "slogan:go-home"),
        (PATTERN_SLOGAN_PROJECT_HOMECOMING, "slogan:project-homecoming"),
        (PATTERN_SLOGAN_MAGA, "slogan:maga"),
        (PATTERN_SLOGAN_MAHA, "slogan:maha"),
        (PATTERN_SLOGAN_MASA, "slogan:masa"),
        (PATTERN_SLOGAN_AMERICA_FIRST, "slogan:america-first"),
        (PATTERN_SLOGAN_GOLDEN_AGE, "slogan:golden-age"),
        (PATTERN_SLOGAN_SAVE_AMERICA, "slogan:save-america"),
        (PATTERN_SLOGAN_LAW_AND_ORDER, "slogan:law-and-order"),
        (PATTERN_SLOGAN_PEACE_THROUGH_STRENGTH, "slogan:peace-through-strength"),
        (PATTERN_SLOGAN_PROMISES_KEPT, "slogan:promises-kept"),
        (PATTERN_SLOGAN_MASS_DEPORTATION, "slogan:mass-deportation"),
        (PATTERN_SLOGAN_MOST_SECURE_BORDER, "slogan:most-secure-border"),
        (PATTERN_SLOGAN_CATCH_RELEASE, "slogan:catch-release"),
```

```
1584:         (PATTERN_SLOGAN_FIND_AND_KILL, "slogan:find-and-kill"),
1585:         (PATTERN_SLOGAN_IMPORT_THIRD_WORLD, "slogan:import-third-world"),
```

speaker

The speaker namespace is produced only when a known speaker alias appears near speech context. One pattern is alias followed within 100 characters by a speech/interview/announcement/remarks word. The other is a speech/interview/announcement/remarks word followed within 80 characters by the alias.

Tag	Count	Precise plain-English rule
`speaker:President Trump`	392 (3.9%)	Matches aliases for President Trump only when the alias is within 100 characters before, or 80 characters after, speech/interview/remarks/announcement/quote/briefing context.
`speaker:Vice President Vance`	31 (0.3%)	Matches aliases for Vice President Vance only when the alias is within 100 characters before, or 80 characters after, speech/interview/remarks/announcement/quote/briefing context.
`speaker:Secretary Mullin`	22 (0.2%)	Matches aliases for Secretary Mullin only when the alias is within 100 characters before, or 80 characters after, speech/interview/remarks/announcement/quote/briefing context.
`speaker:Tom Homan`	11 (0.1%)	Matches aliases for Tom Homan only when the alias is within 100 characters before, or 80 characters after, speech/interview/remarks/announcement/quote/briefing context.
`speaker:First Lady Melania Trump`	6 (0.1%)	Matches aliases for First Lady Melania Trump only when the alias is within 100 characters before, or 80 characters after, speech/interview/remarks/announcement/quote/briefing context.
`speaker:Stephen Miller`	3 (0.0%)	Matches aliases for Stephen Miller only when the alias is within 100 characters before, or 80 characters after, speech/interview/remarks/announcement/quote/briefing context.

Code text

scripts/tag_lexical.py:1106-1139

```
1106:     r"say|says|said|speak(?:s|ing)?|spoke|remark(?:s|ed)?|brief(?:s|ed|ing)?|"
1107:     r"join(?:s|ed|ing)?|interview(?:s|ed|ing)?|sat\s+down|quote(?:s|d)?)"
1108: )
1109: SPEAKER_NOUN_CONTEXT = (
1110:     r"(?:remarks?|speech|address|statement|announcement|interview|quote|"
1111:     r"press\s+(?:conference|briefing|gaggle)|briefing)"
1112: )
1113: SPEAKER_ALIASES: tuple[tuple[str, str], ...] = (
1114:     (
1115:         "First Lady Melania Trump",
1116:         r"(?:First\s+Lady\s+)?Melania\s+Trump|FLOTUS",
1117:     ),
1118:     (
1119:         "Secretary Mullin",
1120:         r"Secretary\s+Mullin|Sec\.?\s+Mullin|@?SecMullinDHS",
1121:     ),
1122:     (
1123:         "President Trump",
1124:         r"President\s+(?:Donald\s+J\.?\s+)?Trump|Donald\s+Trump|@?POTUS|@?realDonaldTrump",
1125:     ),
1126:     (
1127:         "Vice President Vance",
1128:         r"Vice\s+President\s+(?:JD\s+|J\.D\.\s+)?Vance|VP\s+Vance|J\.?D\.?\s+Vance|@?VP",
1129:     ),
1130:     (
1131:         "Tom Homan",
1132:         r"Tom\s+Homan|Thomas\s+D\.?\s+Homan|@?RealTomHoman",
1133:     ),
1134:     (
1135:         "Stephen Miller",
1136:         r"Stephen\s+Miller|@?StephenM",
1137:     ),
1138:     (
1139:         "Gregory Bovino",
```

scripts/tag_lexical.py:2202-2217

```
2202:     """Return speaker tags only when a named official is tied to speech."""
2203:     out: list[tuple[str, tuple[int, int]]] = []
2204:     for canonical, alias in SPEAKER_ALIASES:
2205:         patterns = (
2206:             rf"(?<!\w)({alias})(?!\w).{{0,100}}\b(?:{SPEAKER_ACTION_CONTEXT}|{SPEAKER_NOUN_CONTEXT})\b",
2207:             rf"\b(?:{SPEAKER_ACTION_CONTEXT}|{SPEAKER_NOUN_CONTEXT})\b"
2208:             rf".{{0,80}}\b(?:by|from|with|of|featuring|:)?\s*({alias})(?!\w)",
2209:         )
2210:         for pattern in patterns:
2211:             match = re.search(pattern, text, re.I)
2212:             if match:
2213:                 out.append((f"speaker:{canonical}", match.span(1)))
2214:                 break
2215:     return out
2216:
2217:
```

state

The state namespace is produced by a validated '<place>, <state>' extraction. The captured state must resolve against the U.S. state vocabulary.

Tag	Count	Precise plain-English rule
state:Texas	133 (1.3%)	Emitted as state:Texas when '<place>, Texas' is captured and the state resolves against the U.S. state vocabulary.
state:California	89 (0.9%)	Emitted as state:California when '<place>, California' is captured and the state resolves against the U.S. state vocabulary.
state:New-York	57 (0.6%)	Emitted as state:New-York when '<place>, New York' is captured and the state resolves against the U.S. state vocabulary.
state:Florida	56 (0.6%)	Emitted as state:Florida when '<place>, Florida' is captured and the state resolves against the U.S. state vocabulary.
state:Virginia	46 (0.5%)	Emitted as state:Virginia when '<place>, Virginia' is captured and the state resolves against the U.S. state vocabulary.
state:North-Carolina	23 (0.2%)	Emitted as state:North-Carolina when '<place>, North Carolina' is captured and the state resolves against the U.S. state vocabulary.
state:Utah	22 (0.2%)	Emitted as state:Utah when '<place>, Utah' is captured and the state resolves against the U.S. state vocabulary.
state:Illinois	21 (0.2%)	Emitted as state:Illinois when '<place>, Illinois' is captured and the state resolves against the U.S. state vocabulary.
state:Tennessee	16 (0.2%)	Emitted as state:Tennessee when '<place>, Tennessee' is captured and the state resolves against the U.S. state vocabulary.
state:Minnesota	15 (0.1%)	Emitted as state:Minnesota when '<place>, Minnesota' is captured and the state resolves against the U.S. state vocabulary.
state:Washington	15 (0.1%)	Emitted as state:Washington when '<place>, Washington' is captured and the state resolves against the U.S. state vocabulary.
state:New-Jersey	14 (0.1%)	Emitted as state:New-Jersey when '<place>, New Jersey' is captured and the state resolves against the U.S. state vocabulary.
state:Colorado	13 (0.1%)	Emitted as state:Colorado when '<place>, Colorado' is captured and the state resolves against the U.S. state vocabulary.
state:Oregon	13 (0.1%)	Emitted as state:Oregon when '<place>, Oregon' is captured and the state resolves against the U.S. state vocabulary.
state:Pennsylvania	13 (0.1%)	Emitted as state:Pennsylvania when '<place>, Pennsylvania' is captured and the state resolves against the U.S. state vocabulary.
state:Arizona	12 (0.1%)	Emitted as state:Arizona when '<place>, Arizona' is captured and the state resolves against the U.S. state vocabulary.
state:Maryland	12 (0.1%)	Emitted as state:Maryland when '<place>, Maryland' is captured and the state resolves against the U.S. state vocabulary.
state:Massachusetts	12 (0.1%)	Emitted as state:Massachusetts when '<place>, Massachusetts' is captured and the state resolves against the U.S. state vocabulary.
state:Connecticut	9 (0.1%)	Emitted as state:Connecticut when '<place>, Connecticut' is captured and the state resolves against the U.S. state vocabulary.
state:Georgia	9 (0.1%)	Emitted as state:Georgia when '<place>, Georgia' is captured and the state resolves against the U.S. state vocabulary.
state:Louisiana	7 (0.1%)	Emitted as state:Louisiana when '<place>, Louisiana' is captured and the state resolves against the U.S. state vocabulary.
state:Michigan	7 (0.1%)	Emitted as state:Michigan when '<place>, Michigan' is captured and the state resolves against the U.S. state vocabulary.
state:South-Carolina	7 (0.1%)	Emitted as state:South-Carolina when '<place>, South Carolina' is captured and the state resolves against the U.S. state vocabulary.
state:Missouri	6 (0.1%)	Emitted as state:Missouri when '<place>, Missouri' is captured and the state resolves against the U.S. state vocabulary.
state:Nevada	6 (0.1%)	Emitted as state:Nevada when '<place>, Nevada' is captured and the state resolves against the U.S. state vocabulary.
state:New-Hampshire	6 (0.1%)	Emitted as state:New-Hampshire when '<place>, New Hampshire' is captured and the state resolves against the U.S. state vocabulary.
state:Wisconsin	6 (0.1%)	Emitted as state:Wisconsin when '<place>, Wisconsin' is captured and the state resolves against the U.S. state vocabulary.
state:Alabama	5 (0.0%)	Emitted as state:Alabama when '<place>, Alabama' is captured and the state resolves against the U.S. state vocabulary.
state:Idaho	5 (0.0%)	Emitted as state:Idaho when '<place>, Idaho' is captured and the state resolves against the U.S. state vocabulary.
state:Iowa	5 (0.0%)	Emitted as state:Iowa when '<place>, Iowa' is captured and the state resolves against the U.S. state vocabulary.
state:Vermont	5 (0.0%)	Emitted as state:Vermont when '<place>, Vermont' is captured and the state resolves against the U.S. state vocabulary.

Tag	Count	Precise plain-English rule
state:Indiana	4 (0.0%)	Emitted as state:Indiana when '<place>, Indiana' is captured and the state resolves against the U.S. state vocabulary.
state:Kansas	4 (0.0%)	Emitted as state:Kansas when '<place>, Kansas' is captured and the state resolves against the U.S. state vocabulary.
state:Mississippi	4 (0.0%)	Emitted as state:Mississippi when '<place>, Mississippi' is captured and the state resolves against the U.S. state vocabulary.
state:Kentucky	3 (0.0%)	Emitted as state:Kentucky when '<place>, Kentucky' is captured and the state resolves against the U.S. state vocabulary.
state:New-Mexico	3 (0.0%)	Emitted as state:New-Mexico when '<place>, New Mexico' is captured and the state resolves against the U.S. state vocabulary.
state:Oklahoma	3 (0.0%)	Emitted as state:Oklahoma when '<place>, Oklahoma' is captured and the state resolves against the U.S. state vocabulary.
state:Hawaii	2 (0.0%)	Emitted as state:Hawaii when '<place>, Hawaii' is captured and the state resolves against the U.S. state vocabulary.
state:Ohio	2 (0.0%)	Emitted as state:Ohio when '<place>, Ohio' is captured and the state resolves against the U.S. state vocabulary.
state:Alaska	1 (0.0%)	Emitted as state:Alaska when '<place>, Alaska' is captured and the state resolves against the U.S. state vocabulary.
state:Arkansas	1 (0.0%)	Emitted as state:Arkansas when '<place>, Arkansas' is captured and the state resolves against the U.S. state vocabulary.
state:Delaware	1 (0.0%)	Emitted as state:Delaware when '<place>, Delaware' is captured and the state resolves against the U.S. state vocabulary.
state:Maine	1 (0.0%)	Emitted as state:Maine when '<place>, Maine' is captured and the state resolves against the U.S. state vocabulary.
state:Nebraska	1 (0.0%)	Emitted as state:Nebraska when '<place>, Nebraska' is captured and the state resolves against the U.S. state vocabulary.
state:North-Dakota	1 (0.0%)	Emitted as state:North-Dakota when '<place>, North Dakota' is captured and the state resolves against the U.S. state vocabulary.
state:South-Dakota	1 (0.0%)	Emitted as state:South-Dakota when '<place>, South Dakota' is captured and the state resolves against the U.S. state vocabulary.

Code text

scripts/tag_lexical.py:233-287

```
233:
234: US_STATES: tuple[str, ...] = (
235:     "Alabama",
236:     "Alaska",
237:     "Arizona",
238:     "Arkansas",
239:     "California",
240:     "Colorado",
241:     "Connecticut",
242:     "Delaware",
243:     "Florida",
244:     "Georgia",
245:     "Hawaii",
246:     "Idaho",
247:     "Illinois",
248:     "Indiana",
249:     "Iowa",
250:     "Kansas",
251:     "Kentucky",
252:     "Louisiana",
253:     "Maine",
254:     "Maryland",
255:     "Massachusetts",
256:     "Michigan",
257:     "Minnesota",
258:     "Mississippi",
259:     "Missouri",
260:     "Montana",
261:     "Nebraska",
262:     "Nevada",
263:     "New Hampshire",
264:     "New Jersey",
265:     "New Mexico",
266:     "New York",
267:     "North Carolina",
268:     "North Dakota",
269:     "Ohio",
270:     "Oklahoma",
271:     "Oregon",
272:     "Pennsylvania",
273:     "Rhode Island",
274:     "South Carolina",
275:     "South Dakota",
276:     "Tennessee",
277:     "Texas",
278:     "Utah",
279:     "Vermont",
280:     "Virginia",
281:     "Washington",
282:     "West Virginia",
283:     "Wisconsin",
```

```
 284:     "Wyoming",
 285: )
 286: STATE_BY_LOWER: dict[str, str] = {s.lower(): s for s in US_STATES}
 287:
```

scripts/tag_lexical.py:1229-1230

```
1229: # "<Place>, <State>" — anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b{_PLACE},\s+({_PLACE})\b")
```

scripts/tag_lexical.py:1703-1708

```
1703:     # state:<NAME> — the "<City>, <State>" pattern, validated.
1704:     for m in PATTERN_STATE_CANDIDATE.finditer(text):
1705:         resolved = _resolve_vocab(m.group(1) or "", STATE_BY_LOWER)
1706:         if resolved:
1707:             add(f"state:{_normalize_state(resolved)}", span=m.span(1))
1708:
```

scripts/tag_lexical.py:2046-2051

```
2046:     if low in ("usa", "united states"):
2047:         return "United-States"
2048:     if low == "uk":
2049:         return "United-Kingdom"
2050:     if low == "uae":
2051:         return "United-Arab-Emirates"
```

subject

The subject namespace is produced by direct subject regexes plus deterministic composites. subject:enforcement-op is added only when action:detention and theme:criminal have already both fired.

Tag	Count	Precise plain-English rule
subject:enforcement-op	1,351 (13.5%)	Composite rule: action:detention and theme:criminal have both already fired.
subject:cbp-home-app	189 (1.9%)	Matches CBP Home App, CBP Home, DHS.GOV/CBPHOME, or DHS/CBP URLs containing cbp-home/cbphome/projecthomecoming.
subject:angel-family	54 (0.5%)	Matches angel family/families/mom/dad/parent/mother/father/wife/husband/son/daughter/child.
subject:native-born-citizen	3 (0.0%)	Matches native-born citizens/Americans/U.S. citizens/people/workers/taxpayers.
subject:celebrity	2 (0.0%)	Matches Sydney Sweeney, a celebrity-title phrase followed by a capitalized full name, or a capitalized full name followed by actress/influencer/celebrity/pop star/movie star.
subject:detainee	1 (0.0%)	No deterministic lexical emitter in scripts/tag_lexical.py; this appears only if imported from a vetted sidecar/manual tag source and normalized.
subject:official	1 (0.0%)	No deterministic lexical emitter in scripts/tag_lexical.py; this appears only if imported from a vetted sidecar/manual tag source and normalized.

Code text

scripts/tag_lexical.py:780-804

```
 780: PATTERN_SUBJECT_CBP_HOME_APP = _compile(
 781:     r"\b@?CBP\s+Home\s+App\b"
 782:     r"|\b@?CBP\s+Home\b"
 783:     r"|\bDHS\.?GOV/CBPHOME\b"
 784:     r"|\b(?:dhs|cbp)\.gov/(?:cbp[-_]?home|projecthomecoming)\b"
 785: )
 786: PATTERN_THEME_CBP_HOME = _compile(
 787:     r"\bCBP\s+Home\b"
 788:     r"|\bProject\s+Homecoming\b"
 789:     r"|\bself[- ]deport(?:ed|ing|ation)?\b"
 790:     r"|\bfree\s+(?:flight|plane\s+ticket|ticket)\s+home\b"
 791:     r"|\bcomplimentary\s+plane\s+ticket\s+home\b"
 792:     r"|\bexit\s+bonus\b"
 793:     r"|\btake\s+control\s+of\s+(?:your|their|his|her)\s+departure\b"
 794:     r"|\b(?:illegal\s+aliens?|aliens?|migrants?|CBP\s+Home|self[- ]deport|deport)\b"
 795:     r"[\s\S]{0,120}\bgo\s+home\b"
 796:     r"|\bgo\s+home\b[\s\S]{0,120}\b"
 797:     r"(?:illegal\s+aliens?|aliens?|migrants?|CBP\s+Home|self[- ]deport|deport)\b"
 798: )
 799: PATTERN_SUBJECT_CELEBRITY = _compile(
 800:     r"\bSydney\s+Sweeney\b"
 801:     r"|\b(?:actress|influencer|celebrity|pop\s+star|movie\s+star|Hollywood\s+star)"
 802:     r"\s+[A-Z][a-z]+(?:\s+[A-Z][a-z]+)+\b"
 803:     r"|\b[A-Z][a-z]+(?:\s+[A-Z][a-z]+)+,\s+(?:an?\s+)?"
 804:     r"(?:actress|influencer|celebrity|pop\s+star|movie\s+star)\b"
```

scripts/tag_lexical.py:1209-1216

```
1209: PATTERN_ANGEL_FAMILY = _compile(
1210:     r"\bangel (?:famil(?:y|ies)|mom|dad|parent|mother|father|wife|husband|son|daughter|child)\b"
1211: )
1212: PATTERN_NATIVE_BORN_CITIZEN = _compile(
1213:     r"\bnative[- ]born\s+(?:citizens?|americans?|u\.?s\.?\s+citizens?|people|workers|taxpayers)\b"
1214: )
1215: # Place names are captured case-insensitively (so "from mexico" is caught,
1216: # not just "from Mexico") after a small set of prepositions. The optional
```

scripts/tag_lexical.py:1717-1725

```
1717:     # subject:enforcement-op heuristic from the deterministic rules above:
1718:     # if both action:detention and theme:criminal fire, that's strong enough
1719:     # to set this without tentative.
1720:     if any(e["tag"] == "action:detention" for e in entries) and any(
1721:         e["tag"] == "theme:criminal" for e in entries
```

```
1722:     ):
1723:         add("subject:enforcement-op")
1724:
1725:     # video:<kind> + video:duration-bucket — only when a video is attached.
```

theme

The theme namespace is produced by rhetorical-frame regexes and guard functions. Those helpers handle expletive suppression for religion, context windows for coded nativism, and context windows for martyrdom.

Tag	Count	Precise plain-English rule
`theme:criminal`	2,782 (27.8%)	Matches criminal illegal alien(s), illegal alien(s), criminal alien(s), aggravated felon(s), or convicted for/of.
`theme:directive`	420 (4.2%)	Matches sentence-start imperatives: Apply, Report, Call, Visit, Leave, Self-deport, Go to, Submit, Sign up, Tip, Click, Tap, See, or Read.
`theme:religion`	329 (3.3%)	Matches religion/religious, faith/faithful/faith-based, worship/church/synagogue/mosque/temple/chaplain, prayer/pray/praying/prayed, bless/blessed/blessing, God/Lord/Jesus/Christ/Christian, Bible/biblical/scripture. Suppresses common expletives within 18 characters.
`theme:martyrdom`	202 (2.0%)	Matches Angel Family/family-member language; memorial/victim/fallen/killed/murdered language within 180 characters of honor/remember/commemorate/mourn/never forget/say their names in either order; would still be alive/life was taken/preventable tragedy/line of duty/gave everything. This is our why also matches directly for tracked accounts.
`theme:civil-disturbance`	85 (0.8%)	Matches riot/rioters/rioting, civil disturbance/unrest, mass unrest, violent protest(s), anti-ICE riot/protest, violent demonstrators, street violence, or mob violence.
`theme:homeland`	67 (0.7%)	Matches our/the/this/America's Homeland; protect/secure/defend/safeguard Homeland; or safe/secure Homeland.
`theme:nativism`	30 (0.3%)	Matches native-born/American-born people, foreign-born/foreign workers with displacement/flood/cheap/replacement/betrayal/American-worker context within 140 characters, globalism has failed/Americanism will prevail, forefathers, birthright-as-stolen language, or inheritance language within 160 characters of nation/citizenship/Homeland/birthright/immigration context.
`theme:transgender`	23 (0.2%)	Matches transgender, gender ideology/identity/affirming/transition/dysphoria, biological/transgender male/female/man/woman/boy/girl language, men/boys in women's/girls' sports, women/girls sports exclusion/protection language, or Title IX within 80 characters of transgender/gender/sports context.
`theme:pop-culture-reference`	5 (0.0%)	Matches Sydney Sweeney; American Eagle within 100 characters of ICE/DHS/CBP/deport/illegal alien/border in either order; or good genes within 100 characters of jeans/ICE/DHS/CBP/deport/border/illegal alien. Also implied by parody matches.

Code text

scripts/tag_lexical.py:657-725

```
657: PATTERN_THEME_HOMELAND = re.compile(
658:     r"\b(?i:our|the|this|america's)\s+Homeland\b"
659:     r"|\b(?i:protect|protecting|secure|securing|defend|defending|safeguard|safeguarding)"
660:     r"\s+(?:(?i:our|the|this|america's)\s+)?Homeland\b"
661:     r"|\b(?i:safe|secure)\s+Homeland\b"
662: )
663: PATTERN_THEME_NATIVISM = _compile(
664:     r"\bnative[- ]born\s+(?:americans?|workers?|citizens?)\b"
665:     r"|\bamerican[- ]born\s+(?:americans?|workers?|citizens?)\b"
666:     r"|\bforeign[- ]born\s+workers?\b"
667:     r"|\bforeign\s+(?:workers?|labor)\b.{0,140}\b(?:flood|cheap|displac|replac|"
668:     r"betray|job market|american\s+(?:workers?|jobs?)|americans?\s+first)\b"
669:     r"|\b(?:american\s+(?:workers?|jobs?)|americans?\s+first|job market)\b.{0,140}"
670:     r"\b(?:foreign\s+(?:workers?|labor)|foreign[- ]born\s+workers?)\b"
671:     r"|\bglobalism has failed\b|\bamericanism will prevail\b"
672:     r"|\b(?:our\s+)?forefathers?\b"
673: )
674: PATTERN_NATIVISM_INHERITANCE = _compile(r"\b(?:inheritors?|inheritance|inherit(?:ed|ing)?)\b")
675: PATTERN_NATIVISM_INHERITANCE_CONTEXT = _compile(
676:     r"\b(?:american|america|nation|national|citizens?|native[- ]born|foreign[- ]born|"
677:     r"immigrants?|migrants?|aliens?|homeland|heritage|birthright|forefathers?|"
678:     r"founders?|ancestors?|descendants?|civilization|our\s+people|our\s+country|"
679:     r"our\s+nation|american\s+(?:workers?|jobs?))\b"
680: )
681: PATTERN_THEME_CHRISTIANITY = _compile(
682:     r"\bchristian(?:ity|s)?\b"
683:     r"|\bjudeo[- ]christian\b"
684:     r"|\bchristian\s+(?:faith|values?|church|churches|heritage|nation)\b"
685:     r"|\bjesus(?:\s+christ)?\b"
686:     r"|\bchrist\s+(?:is\s+king|the\s+king|our\s+lord)\b"
687:     r"|\b(?:bible|biblical|scripture|scriptural)\b"
688:     r"|\b(?:[1-3]\s+)?(?:Genesis|Exodus|Leviticus|Numbers|Deuteronomy|Joshua|Judges|Ruth|"
689:     r"Samuel|Kings|Chronicles|Ezra|Nehemiah|Esther|Job|Psalms?|Proverbs|Ecclesiastes|"
690:     r"Song\s+of\s+Songs|Isaiah|Jeremiah|Lamentations|Ezekiel|Daniel|Hosea|Joel|Amos|"
691:     r"Obadiah|Jonah|Micah|Nahum|Habakkuk|Zephaniah|Haggai|Zechariah|Malachi|Matthew|"
692:     r"Mark|Luke|John|Acts|Romans|Corinthians|Galatians|Ephesians|Philippians|"
693:     r"Colossians|Thessalonians|Timothy|Titus|Philemon|Hebrews|James|Peter|Jude|"
694:     r"Revelation)\s+\d{1,3}:\d{1,3}(?:[-\u2013]\d{1,3})?\b"
695:     r"|\bin\s+jesus(?:'s|')?\s+name\b"
696: )
697: PATTERN_THEME_RELIGION = _compile(
698:     r"\breligio(?:n|us)\b"
699:     r"|\bfaith(?:ful|s|[- ]based)?\b"
700:     r"|\b(?:worship|church(?:es)?|synagogue|mosque|temple|chaplain)\b"
701:     r"|\b(?:prayer|prayers|pray|praying|prayed)\b"
702:     r"|\b(?:bless|blessed|blessing|blessings)\b"
703:     r"|\b(?:god|lord|jesus|christ|christian(?:ity|s)?|judeo[- ]christian)\b"
704:     r"|\b(?:bible|biblical|scripture|scriptural)\b"
```

```
)
PATTERN_THEME_RELIGION_EXPLETIVE = _compile(
    r"\b(?:oh\s+my\s+god|omg|god\s*damn(?:ed|it)?|goddamn(?:ed)?|"
    r"for\s+god'?s\s+sake|good\s+lord|thank\s+god)\b"
)
RELIGION_EXPLETIVE_WINDOW_CHARS = 18
PATTERN_THEME_TRANSGENDER = _compile(
    r"\btransgender\b"
    r"|\bgender[- ](?:ideology|identity|affirming|transition|dysphoria)\b"
    r"|\b(?:biological|transgender)\s+"
    r"(?:males?|females?|man|woman|men|women|boys?|girls?)\b"
    r"|\b(?:men|males|boys)\s+in\s+(?:women[\u2019']?s|girls[\u2019']?)\s+sports\b"
    r"|\b(?:men|males|boys)\s+(?:competing|playing|participating)\s+"
    r"(?:in|against|with)\s+(?:women|girls|female\s+athletes?)\b"
    r"|\b(?:women[\u2019']?s|girls[\u2019']?)\s+sports\s+(?:are\s+)?"
    r"for\s+(?:women|girls|female\s+athletes?)\b"
    r"|\b(?:protect|save|defend)\s+(?:women[\u2019']?s|girls[\u2019']?)\s+sports\b"
    r"|\btitle\s+ix\b.{0,80}\b(?:transgender|gender|women[\u2019']?s\s+sports|girls[\u2019']?\s+sports)\b"
    r"|\b(?:transgender|gender|women[\u2019']?s\s+sports|girls[\u2019']?\s+sports)\b.{0,80}\btitle\s+ix\b"
)
PATTERN_THEME_CIVIL_DISTURBANCE = _compile(
```

scripts/tag_lexical.py:765-779

```
PATTERN_MARTYRDOM_WHY = _compile(
    r"\bthis\s+is\s+our\s+why\b"
    r"|\bthey\s+are\s+our\s+why\b"
    r"|\byou\s+are\s+why\s+we\s+fight\b"
)
PATTERN_MARTYRDOM_CONTEXT = _compile(
    r"\bangel\s+(?:famil(?:y|ies)|mom|dad|parent|mother|father|wife|husband|son|daughter|child)\b"
    r"|\b(?:honou?r|remember|commemorate|mourn|never\s+forget|say\s+their\s+names?)\b"
    r".{0,180}\b(?:victims?|fallen|killed|murdered|lives?|life|names?|memory|heroes|families)\b"
    r"|\b(?:victims?|fallen|killed|murdered|lives?|life|names?|memory|heroes|families)\b"
    r".{0,180}\b(?:honou?r|remember|commemorate|mourn|never\s+forget|say\s+their\s+names?)\b"
    r"|\b(?:would\s+still\s+be\s+alive|life\s+was\s+taken|lives?\s+were\s+taken|"
    r"preventable\s+(?:murder|death|tragedy)|killed\s+in\s+the\s+line\s+of\s+duty|"
    r"gave\s+everything)\b"
)
```

scripts/tag_lexical.py:1777-1845

```
    for match in PATTERN_THEME_RELIGION.finditer(text):
        start, end = match.span()
        window = text[
            max(0, start - RELIGION_EXPLETIVE_WINDOW_CHARS) : min(
                len(text), end + RELIGION_EXPLETIVE_WINDOW_CHARS
            )
        ]
        if PATTERN_THEME_RELIGION_EXPLETIVE.search(window):
            continue
        return match
    return None


def _coded_nativism_match(text: str, entries: list[dict[str, Any]]) -> re.Match[str] | None:
    """Match coded inheritance/inheritor language only with nearby context.

    Standalone "inheritance" can be about taxes or probate. In the corpus,
    though, inheritance/inheritor language next to nation, citizenship,
    Homeland, birthright, American workers, or immigration terms is a
    nativism signal.
    """
    existing = {str(e["tag"]) for e in entries}
    strong_tag_context = any(
        tag in {"subject:native-born-citizen", "theme:homeland", "theme:nativism"}
        or tag.startswith(("action:", "origin:"))
        or tag
        in {
            "agency:ICEgov",
            "agency:CBP",
            "agency:DHSgov",
            "slogan:criminal-illegal-alien",
            "slogan:illegal-alien",
            "topic:immigration",
        }
        for tag in existing
    )
    for match in PATTERN_NATIVISM_INHERITANCE.finditer(text):
        start, end = match.span()
        window = text[max(0, start - 160) : min(len(text), end + 160)]
        if strong_tag_context or PATTERN_NATIVISM_INHERITANCE_CONTEXT.search(window):
            return match
    return None


def _martyrdom_match(
    text: str, account_category: str, entries: list[dict[str, Any]]
) -> re.Match[str] | None:
    """Match victim-commemoration frames used as a moral justification.

    "This is our why" is highly distinctive in the tracked core-account
    corpus, but noisy in public replies. We therefore accept that phrase
    directly for tracked core/government/official accounts and require
    nearby victim/memorial context elsewhere.
    """
    if m := PATTERN_MARTYRDOM_CONTEXT.search(text):
        return m

    why = PATTERN_MARTYRDOM_WHY.search(text)
    if not why:
        return None
    existing = {str(e["tag"]) for e in entries}
    if account_category in {"core", "government", "officials"}:
        return why
    if existing.intersection({"subject:angel-family", "subject:crime-victim", "crime:homicide"}):
        return why
    start, end = why.span()
    window = text[max(0, start - 220) : min(len(text), end + 220)]
    if PATTERN_MARTYRDOM_CONTEXT.search(window):
```

```
1845:         return why
```

topic

The topic namespace is produced by direct topic regexes, parent-topic implications from narrower tags, and the immigration sticky default. For tracked core/government/official accounts, topic:immigration is added unless a non-immigration blocker fires; it is tentative when no explicit immigration signal exists.

Tag	Count	Precise plain-English rule
`topic:immigration`	9,196 (91.9%)	Added when explicit immigration regex matches immigration/immigrant/migrant/asylum/illegal alien/undocumented alien/border patrol/CBP Home/deport/removal; or when narrower immigration tags imply it; or by default for core/government/official accounts unless a weather/disaster/sports/birthday/anniversary blocker or another non-immigration topic prevents it. Tentative when only the account default supports it.
`topic:economy`	1,083 (10.8%)	Matches economy/economic, jobs/job growth, workers/workforce/wages/labor market, employment/unemployment/hiring/manufacturing/business/apprenticeship/small business, taxes/tax cuts, inflation/prices/cost of living, tariffs/trade deficit/supply chain/GDP/stock market/markets. Also implied by policy:worksite-enforcement.
`topic:general`	971 (9.7%)	Added when at least three broad problem-domain regexes match among immigration, crime, economy, military, fraud, security, and costs; also implied by broad slogans such as MAGA/MAHA/America First/Golden Age/Save America/Law and Order/Peace Through Strength/Promises Kept.
`topic:military`	658 (6.6%)	Matches military/armed forces/service members/veterans/troops/soldiers/sailors/airmen/marines/guardsmen, service branches, Pentagon/DoD/VA, combat/battlefield/war zone/deployment, carrier/USS/USNS terms, service academies/cadets/midshipmen, commander-in-chief, or commandant. Also implied by any military:<branch> tag.
`topic:laudatory`	99 (1.0%)	Matches accomplishment-list framing: Promises Made/Kept, Golden Age of America, two or more check-mark bullets, delivering on promises/for Americans, historic/record/biggest/greatest/unprecedented/largest accomplishment/achievement/win/progress/result/deal, list/litany/series of accomplishments, or administration/president/Trump accomplishment framing.

Code text

scripts/tag_lexical.py:413-424

```
413: # tweet, we suppress the default `topic:immigration`. The corpus is
414: # overwhelmingly about immigration enforcement; defaulting to ON is
415: # strictly higher recall than trying to infer relevance from sparse text.
416: NON_IMMIGRATION_PATTERNS: tuple[re.Pattern[str], ...] = (
417:     re.compile(r"\b(weather|hurricane|tornado|earthquake|wildfire)\b", re.I),
418:     re.compile(r"\b(NCAA|Super Bowl|World Series|playoffs?)\b", re.I),
419:     re.compile(r"\b(birthday|anniversary)\b", re.I),
420: )
421:
422: # A "sticky default" applied to every tweet from these account categories
423: # unless a NON_IMMIGRATION_PATTERN matches. See the `topic:immigration`
424: # entry in `config/tag_taxonomy.yaml`.
```

scripts/tag_lexical.py:1855-1909

```
1855:     "action:deportation": ("topic:immigration",),
1856:     "action:self-deportation": ("topic:immigration",),
1857:     "action:report-immigrants": ("topic:immigration",),
1858:     "agency:ICEgov": ("topic:immigration",),
1859:     "agency:CBP": ("topic:immigration",),
1860:     "agency:DHSgov": ("topic:immigration",),
1861:     "agency:HSI_HQ": ("topic:immigration",),
1862:     "agency:USBPChief": ("topic:immigration",),
1863:     "agency:DeptofWar": ("topic:military",),
1864:     "agency:CENTCOM": ("topic:military",),
1865:     "agency:Southcom": ("topic:military",),
1866:     "agency:USCG": ("topic:military",),
1867:     "agency:USArmyNorth": ("topic:military",),
1868:     "agency:USNationalGuard": ("topic:military",),
1869:     "agency:USNorthernCmd": ("topic:military",),
1870:     "theme:criminal": ("topic:immigration",),
1871:     "genre:lineup": ("topic:immigration",),
1872:     "subject:angel-family": ("theme:martyrdom", "topic:immigration"),
1873:     "subject:crime-victim": ("theme:martyrdom", "topic:immigration"),
1874:     "subject:cbp-home-app": ("topic:immigration",),
1875:     "subject:enforcement-op": ("topic:immigration",),
1876:     "policy:border": ("topic:immigration",),
1877:     "policy:cbp-home": ("topic:immigration",),
1878:     "policy:sanctuary-cities": ("topic:immigration",),
1879:     "policy:worksite-enforcement": ("topic:economy", "topic:immigration"),
1880:     "theme:nativism": ("topic:immigration",),
1881:     "theme:pop-culture-reference": ("topic:immigration",),
1882:     "religion:christianity": ("theme:religion",),
1883:     "slogan:criminal-illegal-alien": ("topic:immigration",),
1884:     "slogan:free-ticket-home": ("topic:immigration",),
1885:     "slogan:go-home": ("topic:immigration",),
1886:     "slogan:illegal-alien": ("topic:immigration",),
1887:     "slogan:mass-deportation": ("topic:immigration",),
1888:     "slogan:masa": ("topic:immigration",),
1889:     "slogan:find-and-kill": ("topic:immigration",),
1890:     "slogan:import-third-world": ("topic:immigration",),
1891:     "legal:birthright-citizenship": ("topic:immigration",),
1892:     "genre:parody": ("theme:pop-culture-reference",),
1893:     "slogan:most-secure-border": ("topic:immigration", "policy:border"),
1894:     "slogan:catch-release": ("topic:immigration",),
1895:     "slogan:project-homecoming": ("topic:immigration",),
```

```
    "slogan:maga": ("topic:general",),
    "slogan:maha": ("topic:general",),
    "slogan:america-first": ("topic:general",),
    "slogan:golden-age": ("topic:general",),
    "slogan:save-america": ("topic:general",),
    "slogan:law-and-order": ("topic:general",),
    "slogan:peace-through-strength": ("topic:general",),
    "slogan:promises-kept": ("topic:general",),
    "phrase:immigrant": ("topic:immigration",),
    "phrase:migrant": ("topic:immigration",),
}
INTRINSIC_PARENT_TOPICS_PREFIXES: tuple[tuple[str, str], ...] = (("origin:", "topic:immigration"),)
PATTERN_EXPLICIT_IMMIGRATION_TOPIC = _compile(
    r"\b(immigration|immigrants?|migrants?|asylum|illegal\s+(?:alien|immigrant)s?|"
```

scripts/tag_lexical.py:1936-2041

```
    if PATTERN_EXPLICIT_IMMIGRATION_TOPIC.search(text):
        add("topic:immigration")
    if (
        any(e["tag"] == "policy:worksite-enforcement" for e in entries)
        and "topic:economy" not in existing_tags
    ):
        add("topic:economy")


# Tag names whose presence on a tweet promotes `topic:immigration` from
# a tentative-by-account default to a confirmed-by-text classification.
# Anything that explicitly references the immigration domain (origin
# country pattern, deportation verb, border keyword, ICE/CBP/DHS handle,
# the criminal-alien frame, etc.) clears the bar.
IMMIGRATION_CONFIRMING_PREFIXES: tuple[str, ...] = (
    "action:",
    "origin:",
    "country:",
    "policy:border",
    "policy:sanctuary",
    "policy:worksite",
    "policy:cbp-home",
    "theme:nativism",
    "theme:criminal",
    "shape:",
    "subject:cbp-home-app",
    "subject:enforcement-op",
)
IMMIGRATION_CONFIRMING_EXACT: frozenset[str] = frozenset(
    {
        "agency:ICEgov",
        "agency:CBP",
        "agency:DHSgov",
        "agency:HSI_HQ",
        "agency:USBPChief",
        "slogan:criminal-illegal-alien",
        "slogan:free-ticket-home",
        "slogan:go-home",
        "slogan:illegal-alien",
        "slogan:mass-deportation",
        "slogan:masa",
        "slogan:most-secure-border",
        "slogan:catch-release",
        "slogan:nice",
        "slogan:project-homecoming",
        "slogan:reportrecon",
        "slogan:worst",
        "phrase:immigrant",
        "phrase:migrant",
    }
)
# Last-ditch keyword check — picks up image-heavy / template-light
# tweets that the namespace rules above don't flag but that still
# clearly read as immigration content ("ICE arrested", "illegal alien",
# "the border", standalone "immigration"). Kept narrow on purpose.
PATTERN_IMMIGRATION_KEYWORD = _compile(
    r"\b(immigration|immigrant|migrant|asylum|illegal alien|the border|"
    r"border patrol|ICE\b|CBP\b)\b"
)


def _has_immigration_signal(entries: list[dict[str, Any]], text: str) -> bool:
    """True if the tweet carries any explicit immigration-domain signal.
    Drives the confirmed-vs-tentative split for `topic:immigration`."""
    for e in entries:
        tag = e["tag"]
        if tag in IMMIGRATION_CONFIRMING_EXACT:
            return True
        for pref in IMMIGRATION_CONFIRMING_PREFIXES:
            if tag.startswith(pref):
                return True
    return bool(PATTERN_IMMIGRATION_KEYWORD.search(text))


def _has_non_immigration_topic(entries: list[dict[str, Any]]) -> bool:
    return any(e["tag"].startswith("topic:") and e["tag"] != "topic:immigration" for e in entries)


def _maybe_immigration_default(
    entries: list[dict[str, Any]],
    account_category: str,
    text: str,
    add: Callable[..., None],
) -> None:
    """Apply `topic:immigration` to every tweet from a tracked-tier
    author unless an obvious non-immigration signal blocks it.
    Confirmed when the tweet carries any explicit immigration-domain
    marker; tentative otherwise (image-heavy, template-light tweets
    where the author identity is the only signal we have).

    See the `topic:immigration` entry in `config/tag_taxonomy.yaml` for
    the rationale."""
    if account_category not in IMMIGRATION_DEFAULT_CATEGORIES:
```

```
        return
    has_signal = _has_immigration_signal(entries, text)
    if has_signal:
        add("topic:immigration")
        return
    if any(e["tag"] == "event:palestine" for e in entries):
        return
    for pat in NON_IMMIGRATION_PATTERNS:
        if pat.search(text):
            return
    if not _has_non_immigration_topic(entries):
        add("topic:immigration", tentative=True)

```